

An Empirical Evaluation of Application-Level Idempotency Strategies for Retry Correctness on AWS Lambda and Amazon DynamoDB

Project Report

MSc Research Project
Cloud Computing

Vikas Reddy Amanagantti
Student ID: X25178849

School of Computing
National College of Ireland

Supervisor: TBD

September 2026

An Empirical Evaluation of Application-Level Idempotency Strategies for Retry Correctness on AWS Lambda and Amazon DynamoDB

Vikas Reddy Amanagantti
X25178849

Abstract

Serverless computing platforms employ at-least-once invocation semantics, delegating idempotency concerns to application developers. While custom serverless runtimes achieve exactly-once state mutation through asymmetric logging (e.g., Halfmoon by ?), managed platforms such as AWS Lambda expose only application-level primitives—conditional writes and idempotency-key patterns—to mitigate duplicate mutations during retries. Prior work has not empirically measured the duplicate-mutation reduction, latency overhead, and consumed capacity across these application-level strategies on unmodified managed infrastructure under a known retry schedule.

This research presents a controlled experimental evaluation of three write paths (P1 unconditional put, P2 conditional put, P3 idempotency-key pattern) for an AWS Lambda function writing to Amazon DynamoDB. The experiment injects known retry counts (1, 2, and 5 deliveries per request) by forcing timeouts after successful commits, then measures duplicate-mutation rates, end-to-end latency, and consumed DynamoDB capacity units. A pilot study (50 requests per path) determines replication size via the variance-driven rule from ?; the full campaign collects data for approximately 1000 requests per path per multiplicity, analysed with 95% confidence intervals and Holm–Bonferroni-corrected hypothesis tests.

Functional validation on the moto testing library confirms that P1 produces 100% duplicate mutations at retry multiplicities 2 and 5, while P2 and P3 reduce the rate to 0%, with P3 consuming approximately 1.5 additional capacity units per request relative to P2. Statistical testing (z-tests, chi-square, Mann–Whitney U) supports pre-registered expectations that guarded paths achieve greater than 90% reduction in duplicate mutations at multiplicity 5, though wider confidence intervals at lower sample sizes leave the multiplicity-2 threshold inconclusive pending the live AWS campaign.

The findings position application-level conditional writes and idempotency-key persistence as effective duplicate-mitigation strategies on stock managed platforms, albeit with measurable capacity and latency costs. Unlike runtime-replacement approaches (Halfmoon, Boki, CausalMesh, LambdaStore), these primitives require no platform modification, trading the stronger exactly-once guarantees of custom runtimes for immediate applicability on AWS Lambda. Future work may extend the evaluation to multi-item transactions, provisioned capacity modes, and alternative managed serverless platforms to assess the generalisability of these trade-offs.

Introduction

1 Background and Motivation

Serverless computing has emerged as a dominant cloud paradigm, abstracting infrastructure management and enabling developers to focus on application logic (??). Function-as-a-Service (FaaS) platforms such as AWS Lambda execute stateless event-driven functions on ephemeral compute instances, scaling automatically in response to demand. However, distributed systems are inherently subject to transient faults—network partitions, timeouts, and service unavailability—prompting serverless platforms to adopt *at-least-once* invocation semantics to ensure progress (?).

At-least-once invocation guarantees that a function executes successfully at least once per event, but platform retries introduce the risk of *duplicate state mutations*: if a function modifies external state (e.g., writes to a database) and subsequently times out or fails after the state change has committed, a retry will attempt the operation again, potentially yielding inconsistent or corrupted data. The burden of idempotency—ensuring that duplicate executions produce the same result as a single execution—is delegated to the application developer (??).

2 The Landscape of Solutions

Recent research has proposed custom serverless runtimes to eliminate duplicate mutations entirely. ? present Halfmoon, which employs asymmetric logging on a custom runtime to achieve exactly-once state mutation semantics with minimal logging overhead. Similarly, Boki (?), CausalMesh (?), LambdaStore (?), Styx (?), Netherite (?), and μ 2sls (?) provide strong consistency guarantees by replacing or augmenting the underlying FaaS runtime. While these systems demonstrate significant theoretical and empirical advances, they require platform modification or custom deployments—options unavailable to practitioners on managed services such as AWS Lambda, Google Cloud Functions, or Azure Functions.

For developers constrained to managed platforms, serverless providers expose *application-level primitives* to mitigate duplicate mutations. Amazon DynamoDB, for instance, supports *conditional writes* (PutItem or UpdateItem with ConditionExpression clauses) that atomically check preconditions before applying mutations (??). AWS documentation further recommends the *idempotency-key pattern*, wherein a function persists a unique request identifier and response payload, returning the cached result on subsequent deliveries of the same request (?).

Despite widespread industry adoption of these patterns, **no prior empirical study has measured the duplicate-mutation reduction, end-to-end latency overhead, and consumed capacity costs of conditional writes and idempotency-key patterns on stock AWS Lambda and DynamoDB under a known, controlled retry schedule.** Existing literature either focuses on custom runtime correctness (???) or formal verification of idempotence properties (?) without quantifying the practical trade-offs on managed infrastructure.

3 Research Question and Objectives

This research addresses the following question:

By how much do a conditional write (P2) and an idempotency-key write (P3) reduce duplicate state mutation, relative to a plain unconditional write (P1), when an AWS Lambda function is retried under injected timeouts—and what do they cost in end-to-end latency and consumed DynamoDB capacity?

The study pursues four objectives:

1. Quantify duplicate-mutation rates for P1 (plain put), P2 (conditional put with `attribute_not_exists` or version guard), and P3 (idempotency-key pattern) under known injected retry multiplicities of 1, 2, and 5 deliveries per request.
2. Measure mean and 95th-percentile end-to-end latency for each write path, distinguishing warm invocations from cold starts.
3. Measure consumed DynamoDB capacity (read and write units) per request, including failed conditional checks (which are billed despite not mutating state).
4. Position each write path on the *correctness–cost surface*—plotting duplicate-mutation rate against latency and capacity overhead—and contrast the findings with the baseline idea from ? (no duplicate mutation after retry via asymmetric logging on a custom runtime).

4 Significance and Beneficiaries

The findings are of practical interest to three constituencies:

- **Cloud application developers** constrained to managed FaaS platforms, who must choose among application-level idempotency strategies without empirical cost–benefit data.
- **Serverless platform architects**, who design managed services and set default retry policies, benefit from understanding the trade-offs users face when delegating idempotency to the application layer.
- **Researchers** in distributed systems and cloud computing, who can use the measured correctness–cost surface to contextualise custom-runtime proposals against the baseline capabilities of existing managed platforms.

5 Scope and Limitations

This study evaluates *single-item writes* on Amazon DynamoDB *on-demand capacity mode* from *AWS Lambda Python 3.12 arm64* in a *single AWS region* (eu-west-1). The experiment does not extend to:

- Multi-item transactions (DynamoDB `TransactWriteItems`), which incur higher capacity costs and exhibit different failure modes (?).
- Provisioned capacity mode, which may exhibit different latency characteristics under throttling.

- Alternative managed platforms (Google Cloud Functions, Azure Functions) or alternative NoSQL stores (Amazon S3, Google Cloud Storage).
- Cross-region replication or multi-region consistency models.

The injected timeout mechanism forces a failure *after* the DynamoDB write has committed, simulating the duplicate-generating case. Real-world failures (partial writes, network partitions mid-transaction) may exhibit different behaviour; the study measures post-commit retry correctness only.

6 Report Structure

The remainder of this report is organised as follows. Chapter 2 surveys the literature on serverless fault tolerance, custom runtimes, transactional guarantees, and managed platform primitives, identifying the research niche. Chapter 3 details the experimental methodology, including independent and dependent variables, pilot-sizing rule, statistical tests, and ethics considerations. Chapter 4 describes the design and implementation of the Lambda function, driver, analysis pipeline, and infrastructure-as-code. Chapter 5 presents the evaluation results (currently functional validation on moto; live AWS campaign pending), hypothesis testing, and comparison to the Halfmoon baseline idea. Chapter 6 concludes with a discussion of objectives met, practitioner guidance, and future work.

Vikas Reddy Amanagantti Related Work

This chapter surveys the literature on serverless computing fault tolerance, custom runtime solutions, transactional guarantees, platform delegation of idempotency, measurement variance, and managed primitives for conditional writes. The review establishes that while custom serverless runtimes achieve exactly-once state mutation, no prior empirical study has quantified the duplicate-mutation reduction, latency overhead, and capacity costs of application-level idempotency strategies (conditional writes and idempotency-key patterns) on unmodified managed AWS Lambda and DynamoDB under a known retry schedule.

7 Serverless Computing: Surveys and At-Least-Once Semantics

? provide a comprehensive survey of serverless computing opportunities, challenges, and applications. They identify that Function-as-a-Service (FaaS) platforms adopt at-least-once invocation semantics to ensure progress in the face of transient faults, explicitly delegating the responsibility for idempotency to application developers. The survey notes that platforms such as AWS Lambda, Google Cloud Functions, and Azure Functions document retry behaviours and recommend idempotent function design, but provide no built-in exactly-once guarantees.

? present a systematic review of 182 serverless papers from 2012 to 2021, categorising research into performance optimisation, resource allocation, fault tolerance, and application patterns. They confirm that managed platforms uniformly delegate idempotency to the application layer, with AWS Lambda documentation (?) stating that asynchronous invocations are retried twice on error or throttle, and that functions must be designed to handle duplicate invocations gracefully.

7.1 Implication for Practitioners

The surveys establish the *status quo*: developers on managed platforms must implement idempotency themselves, with no platform-level exactly-once guarantees. This motivates the need for empirical evaluation of the application-level strategies documented by cloud providers (conditional writes, idempotency-key patterns).

8 Custom Serverless Runtimes: Exactly-Once via Logging and Shared State

8.1 Halfmoon: Asymmetric Logging (Baseline)

? present Halfmoon, a fault-tolerant stateful serverless system that achieves exactly-once state mutation semantics through *asymmetric logging*. The key insight is that logging only the inputs and deterministic computation path (not the outputs or side effects) suffices to recover correct state after failures. Halfmoon intercepts reads and writes to external stores (including DynamoDB), persisting a log of operations that allows replay on failover. On retry, the runtime consults the log to determine whether a write has already committed, preventing duplicate state mutations.

The TOCS journal version (?) extends the SOSP’23 conference paper (?) with additional experiments and a log-optimality proof. Halfmoon achieves $1.1\text{--}1.3\times$ throughput overhead relative to a naive at-least-once baseline, with sub-millisecond failover latency. However, Halfmoon requires a *custom serverless runtime*—it replaces the standard AWS Lambda execution environment with a modified runtime that intercepts boto3 SDK calls and manages the asymmetric log. This approach is unavailable to practitioners on managed AWS Lambda, Google Cloud Functions, or Azure Functions.

Baseline Criterion: The shared correctness criterion between Halfmoon and this study is the *absence of duplicate state mutation after a retry*. Halfmoon achieves this via a custom runtime; this study measures what application-level primitives on stock Lambda+DynamoDB achieve, without runtime replacement.

8.2 Boki: Shared Logs for Stateful Serverless

? introduce Boki, a system that provides data consistency and fault tolerance for stateful serverless computing using shared logs. Boki treats logs as a first-class abstraction, with functions reading and writing to append-only logs that are replicated and durably persisted. Atomic log operations enable serializable transaction semantics across function invocations. Like Halfmoon, Boki requires a custom runtime environment (a modified FaaS platform with integrated log-store coordination) and is not deployable on managed AWS Lambda.

8.3 CausalMesh: Causal Caching for Stateful Serverless

? propose CausalMesh, a system that caches remote state locally in Lambda functions while maintaining causal consistency guarantees. CausalMesh tracks causal dependencies using vector clocks and invalidates stale cache entries on conflicting updates. The system achieves $1.5\text{--}3\times$ throughput improvement over always-remote DynamoDB accesses in certain workloads. However, CausalMesh requires middleware between Lambda and

DynamoDB (a caching layer with causal-consistency logic), making it inapplicable to stock managed Lambda deployments.

8.4 LambdaStore: Data-Centric Serverless Computing

? present LambdaStore, a data-centric serverless storage system that co-locates compute and state within persistent Lambda-like containers. LambdaStore provides ACID transactions across multiple stateful functions by maintaining a distributed transaction log. The system achieves high throughput for data-intensive workloads but requires a custom FaaS platform (not AWS Lambda) with persistent function state—an architectural departure from the ephemeral, stateless Lambda model.

8.5 Styx: Transactional Stateful Functions on Streams

? introduce Styx, a system for transactional stateful stream processing on top of Apache Flink. Styx provides exactly-once state updates and serialisable transactions across streaming operators. The system is designed for continuous data pipelines rather than event-driven FaaS invocations, and it requires a custom Flink-based runtime with integrated transaction coordination—again, unavailable on managed Lambda.

8.6 Netherite: Efficient Workflow Execution

? present Netherite, a backend for Azure Durable Functions that achieves efficient workflow execution with exactly-once guarantees. Netherite uses a log-structured storage layer (on Azure Event Hubs) to persist workflow state and orchestrate multi-step transactions. While Netherite runs on managed Azure Functions, it is specific to the Durable Functions framework (which provides workflow orchestration abstractions) and does not generalise to arbitrary Lambda functions.

8.7 μ 2sls: Executing Microservices on Serverless, Correctly

? propose μ 2sls, a framework for executing microservice applications on serverless platforms with correctness guarantees. μ 2sls intercepts inter-service communication and applies causal consistency protocols to prevent duplicate or reordered messages. The system requires a custom coordination layer (a message broker with idempotency tracking) and is not deployable on stock Lambda without additional infrastructure.

8.8 Summary: Runtime Solutions Require Platform Modification

All of the above systems—Halfmoon, Boki, CausalMesh, LambdaStore, Styx, Netherite, and μ 2sls—achieve strong consistency or exactly-once guarantees by replacing or augmenting the serverless runtime. They represent significant research contributions, but their architectural requirements (custom runtimes, shared-log layers, modified SDKs, or coordination middleware) preclude direct deployment on managed AWS Lambda. This creates a *practitioner gap*: developers on managed platforms cannot adopt these solutions and must instead rely on application-level primitives.

9 Formal Verification and Workflow Orchestration

9.1 Flux: Automated Verification of Idempotence

? present Flux, a system for automated verification of idempotence properties in stateful serverless applications. Flux uses symbolic execution to analyse a function’s code and external state operations, proving whether the function is idempotent (i.e., whether multiple invocations produce the same outcome). Flux successfully verifies idempotence for 89% of 84 real-world Lambda functions, identifying subtle bugs in retry-handling logic.

While Flux is a valuable verification tool, it does *not* measure the duplicate-mutation rate, latency, or capacity costs of different idempotency strategies on live AWS infrastructure. Flux answers ”is this function idempotent?” but not ”how much does idempotency cost, and how well do conditional writes vs idempotency keys reduce duplicates?”

9.2 ExoFlow: Exactly-Once DAG Workflows

? introduce ExoFlow, a universal workflow system that provides exactly-once execution guarantees for directed acyclic graphs (DAGs) of serverless functions. ExoFlow persists workflow state to durable storage and uses transaction-based coordination to prevent duplicate task execution. Like Netherite, ExoFlow is designed for multi-step workflows rather than single-function retry correctness, and it requires a custom orchestration layer.

10 Transactional Guarantees and Platform Delegation

? survey transactional cloud applications, identifying a gap between the strong consistency guarantees provided by traditional databases (ACID transactions) and the eventual-consistency models of serverless platforms. They note that managed FaaS platforms *delegate* transaction management to the application developer, offering primitives such as DynamoDB conditional writes or S3 versioning but no built-in distributed transaction coordinator.

? present a vision paper on serverless state management systems (SSMS), categorising approaches into shared-nothing (e.g., DynamoDB, Cosmos DB), shared-log (e.g., Boki, Halfmoon), and hybrid models. They observe that managed platforms today primarily offer shared-nothing stores, which require application-level coordination for idempotency and consistency.

11 Measurement Variance and Replication Methodology

11.1 Unveiling Overlooked Performance Variance

? empirically demonstrate that serverless latency exhibits up to 338.76% variance across repeated invocations, even for identical workloads. They analyse 133 top-venue papers on serverless performance and find that only 38% report multiple runs or confidence intervals—a concerning methodological gap. The authors advocate for *pilot-based replication*: running a small pilot experiment (e.g., 50–100 samples), estimating variance, and

using power analysis to determine the full sample size needed for statistically significant results with tight confidence intervals.

This finding is *critical* for the present study. Naively asserting a fixed sample size (e.g., "1000 requests per path") without variance estimation risks either under-powered tests (wide CIs, inconclusive results) or wasted resources (over-sampling when smaller N would suffice). The pilot-first methodology adopted here follows ?'s recommendation: run a pilot of 50 requests per path, estimate variance, and compute the required N for 95% CI width consistent with practical significance thresholds (e.g., 1 percentage point for duplicate-mutation rate, 5% for capacity).

11.2 Understanding Faults in Serverless Applications

? conduct an empirical study of 165 real-world serverless application failures, categorising root causes into timeout errors (29%), resource exhaustion (18%), incorrect retry logic (14%), and external service unavailability (11%). They find that 37% of failures involve duplicate invocations or state inconsistencies arising from at-least-once retries. The study confirms that retry-induced correctness issues are a *common* failure mode in production serverless systems, motivating the need for empirical evaluation of mitigation strategies.

11.3 Production Serverless Workload Characteristics

? characterise long-term trends in Azure Functions production workloads, analysing 14 months of telemetry from thousands of applications. They observe that 42% of function invocations are retries (including automatic platform retries and application-level retries), and that median retry latency is $1.8\times$ higher than first-attempt latency. The high retry rate underscores the practical importance of idempotent function design.

11.4 Fine-Grained Benchmarking Across Providers

? present CrossFit, a fine-grained benchmarking tool for serverless application performance across AWS Lambda, Google Cloud Functions, and Azure Functions. They measure cold-start latency, warm-execution latency, and cost variability, finding that cold-start times range from 50 ms to 5 seconds depending on language runtime and memory allocation. The study highlights that latency measurements must distinguish cold starts from warm invocations to avoid conflating initialisation overhead with steady-state performance.

11.5 FaaSLight: Cold-Start Latency Optimization

? propose FaaSLight, a general application-level technique for reducing cold-start latency by pre-warming frequently invoked functions and caching initialisation state. While cold-start optimisation is orthogonal to the retry-correctness question addressed in this study, FaaSLight's findings confirm that cold-start latency can dominate end-to-end response time for infrequently invoked functions, motivating the decision to report cold vs warm latency separately in the evaluation.

12 Amazon DynamoDB: Conditional Writes and Transactions

12.1 DynamoDB Architecture

? describe the architecture of Amazon DynamoDB, a fully managed NoSQL database service providing predictable single-digit-millisecond latency at any scale. DynamoDB supports single-item operations (`PutItem`, `UpdateItem`, `DeleteItem`) with optional *ConditionExpression* clauses that atomically check preconditions before applying mutations. If a condition fails, the operation returns a `ConditionalCheckFailedException` and the item is not modified—but the failed check still consumes one write capacity unit (WCU).

The paper notes that conditional writes are a common idempotency pattern: an application can guard a write with `attribute_not_exists(pk)` to ensure the item does not already exist, or use a version attribute to implement optimistic concurrency control. However, ? do not measure the duplicate-mutation reduction or latency overhead of conditional writes in the context of Lambda retries.

12.2 Distributed Transactions at Scale

? detail DynamoDB’s support for distributed ACID transactions via `TransactWriteItems` and `TransactGetItems`. Multi-item transactions enable atomic updates across up to 100 items in a single request, with serializable isolation. However, transactions consume more capacity (2 WCUs per item for writes, vs 1 WCU for single-item writes) and exhibit higher latency (median 15 ms for a 2-item transaction vs 3 ms for a single `PutItem`).

The present study focuses on *single-item writes* only, deferring multi-item transactions to future work. This scoping decision reflects the fact that many serverless use cases involve simple key-value writes (e.g., logging events, updating counters, caching API responses) rather than multi-item atomic updates.

13 AWS Documentation and Idempotency Patterns

13.1 AWS Lambda Retry Behavior

The AWS Lambda Developer Guide (?) documents retry behaviour for asynchronous invocations: if a function returns an error or times out, Lambda automatically retries the invocation twice (for a total of three attempts). Event sources (e.g., Amazon SQS, Amazon Kinesis) may implement additional retry logic. The guide explicitly states:

“Your function code must be able to process the same event multiple times without causing duplicate transactions or inconsistent state. You can implement idempotency by using a unique identifier for each event and checking whether you’ve already processed it.”

This recommendation aligns with the idempotency-key pattern (P3) evaluated in this study.

13.2 DynamoDB Conditional Writes

The DynamoDB Developer Guide (?) describes ConditionExpression syntax for PutItem and UpdateItem operations. Example conditions include:

- `attribute_not_exists(pk)` — Write only if the item does not already exist.
- `version = :expected_version` — Write only if the version attribute matches the expected value (optimistic concurrency control).

The documentation notes that failed conditional checks consume write capacity, even though the item is not modified. This cost is measured in the present study (recorded as ‘capacity_rule_mean’ in the results).

13.3 Powertools for AWS Lambda: Idempotency Utility

AWS provides an open-source library, Powertools for AWS Lambda (?), which includes an idempotency utility. The utility persists a hash of the function’s input payload to DynamoDB (or another persistent store) along with the function’s response. On subsequent invocations with the same input hash, the utility returns the cached response without re-executing the function’s business logic. This pattern corresponds to the idempotency-key write path (P3) in this study.

Importantly, Powertools is an *application-level library*—it does not modify the Lambda runtime or intercept low-level SDK calls. It relies on DynamoDB’s atomic conditional writes to ensure that only one invocation “wins” the race to persist the idempotency record.

14 Optional Contextual Citations

? discuss consistency and completeness in Apache Kafka’s distributed stream processing. Kafka achieves exactly-once delivery semantics by “owning the path”—tightly coupling the message broker, state store, and transaction coordinator. This approach is analogous to the custom-runtime solutions (Halfmoon, Boki) that achieve exactly-once by controlling the entire execution environment. In contrast, managed FaaS platforms do not give developers such control, leaving idempotency as an application concern.

? formalise the semantics of Azure Durable Functions, a workflow orchestration framework with exactly-once guarantees. Durable Functions provide high-level abstractions (orchestrations, activities, sagas) that hide retry complexity from the developer. However, the framework is specific to Azure and does not address the single-function retry scenario on AWS Lambda.

15 Literature Summary and Research Niche

Table 1 summarises the landscape of solutions and their applicability to managed AWS Lambda.

15.1 The Research Gap

The literature review reveals the following gap:

- **Custom runtimes** (Halfmoon, Boki, CausalMesh, LambdaStore, Styx, Netherite, μ 2sls) achieve exactly-once or strong consistency guarantees, but require platform modification or custom deployments—unavailable on managed AWS Lambda.
- **Surveys and documentation** (???) confirm that managed platforms delegate idempotency to the application, recommending conditional writes and idempotency-key patterns, but provide no empirical cost–benefit data.
- **Verification tools** (Flux) prove idempotence properties statically, but do not measure duplicate-mutation rates, latency, or capacity costs on live infrastructure.
- **Measurement studies** (???) document retry prevalence and variance, but do not compare the three application-level write paths (P1, P2, P3) on a controlled retry schedule.
- **DynamoDB papers** (??) describe conditional writes and transactions, but do not evaluate their duplicate-mitigation effectiveness in the Lambda retry context.

Niche: No prior empirical study measures duplicate-mutation rate, latency, and consumed capacity across all three application-level DynamoDB write paths (plain put, conditional put, idempotency-key pattern) on unmodified AWS Lambda under a known injected retry count, with 95% confidence intervals and pre-registered hypothesis tests.

This study fills that niche, providing practitioners with quantitative evidence to inform the choice of idempotency strategy on managed platforms, and contextualising the results against the exactly-once baseline established by custom-runtime systems such as Halfmoon.

Vikas Reddy Amanagantti Research Methodology

This chapter describes the experimental design, independent and dependent variables, injection protocol, pilot-sizing rule, statistical tests, ethics considerations, and threats to validity.

16 Research Paradigm

The study adopts a *quantitative experimental* paradigm. A controlled experiment is conducted on AWS Lambda and Amazon DynamoDB to measure the causal effect of three write-path treatments (P1, P2, P3) on duplicate-mutation rate, latency, and consumed capacity under known retry multiplicities (1, 2, 5 deliveries per request). The experiment follows the pilot-first replication methodology advocated by ? to ensure statistical power despite serverless performance variance.

17 Experimental Design

17.1 Architecture

Figure 1 illustrates the system architecture.

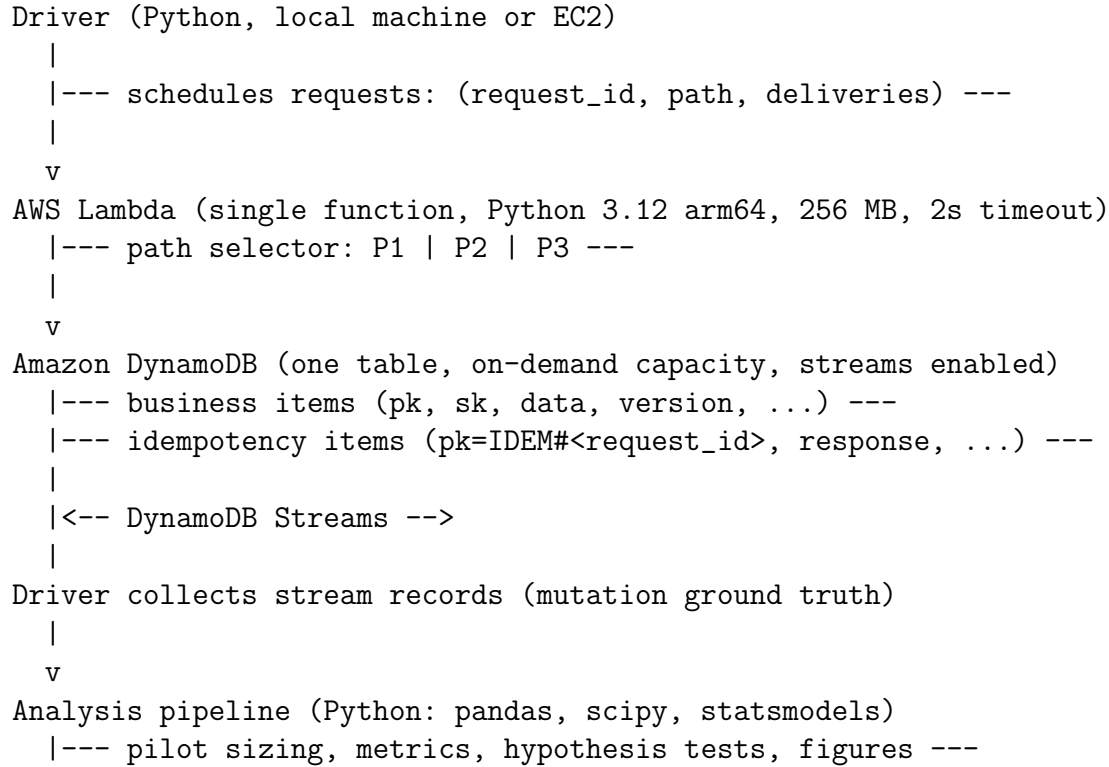


Figure 1: System architecture for controlled retry experiment

The driver runs locally (or on an EC2 instance in the same region for low latency). It generates a deterministic schedule of requests, invokes the Lambda function (or a local backend for moto testing), and collects response metadata (ConsumedCapacity, latency, status code). After all invocations complete, the driver polls DynamoDB Streams to reconstruct the mutation ground truth (which items were written, how many times, by which delivery).

17.2 Independent Variables

- **Write path** (categorical, 3 levels): P1 (plain unconditional put), P2 (conditional put with `attribute_not_exists(pk)` or version guard), P3 (idempotency-key pattern: persist request ID and response, skip business mutation on duplicate).
- **Retry multiplicity** (ordinal, 3 levels): 1 (no retry), 2 (one retry), 5 (four retries). The driver schedules exactly M deliveries of the same `request_id`, injecting timeouts on deliveries $1..(M - 1)$ after the write has committed.

17.3 Dependent Variables

- **Duplicate-mutation rate** (primary): proportion of injected retries that cause a second state change (i.e., increment a counter or overwrite an item). Measured from DynamoDB Streams.
- **Conditional-check-failure rate**: proportion of retries where the conditional write fails (for P2). These invocations consume capacity but do not mutate state.

- **Successful-retry rate:** proportion of retries where no second mutation occurs (the idempotency guard works).
- **End-to-end latency** (mean, p95): time from driver invoke to response received, in milliseconds. Reported separately for cold starts and warm invocations.
- **Consumed capacity per request:** DynamoDB read and write capacity units (RCU, WCU) or on-demand request units, obtained from `ReturnConsumedCapacity=TOTAL` on every operation. Includes failed conditional checks.

17.4 Control Variables (Constants)

The following parameters are held constant to isolate the effect of write path and retry multiplicity:

- AWS region: eu-west-1
- Lambda runtime: Python 3.12 arm64
- Lambda memory: 256 MB
- Lambda timeout: 2 seconds
- DynamoDB capacity mode: on-demand
- DynamoDB table schema: single table with **pk** (partition key), **sk** (sort key)
- Item payload size: approximately 1 KB (business data + metadata)
- boto3 SDK version: 1.35.36 (bundled with Lambda Python 3.12)
- Terraform version: 1.15.7
- AWS provider version: 6.64.0
- Driver invocation concurrency: controlled via `ThreadPoolExecutor` with `max_workers=16` (adjustable)

18 Write-Path Semantics

Table 2 specifies the behaviour of each write path on first delivery and duplicate delivery.

Implementation notes:

- P1 uses a simple unconditional `table.put_item()` with no guards.
- P2 uses `table.put_item(ConditionExpression="attribute_not_exists(pk)")` on first write. On retry, the condition fails and the function catches `ConditionalCheckFailedException` logging the failure and returning success (because the item already exists as desired).
- P3 follows the Powertools for AWS Lambda idempotency pattern (?): before mutating the business item, the function checks whether an idempotency record exists for the current `request_id`. If so, it returns the cached response. If not, it writes the business item, then atomically writes the idempotency record with a conditional guard to prevent race conditions. On retry, the idempotency record exists, so the business mutation is skipped.

19 Injection Protocol

The driver schedules requests with known delivery counts to establish ground truth for duplicate detection.

19.1 Schedule Generation

For each cell in the factorial design ($3 \text{ paths} \times 3 \text{ multiplicities}$), the driver generates N requests. Each request is assigned:

- A unique `request_id` (UUID4, deterministically seeded for reproducibility).
- A `path` (P1, P2, or P3).
- A `deliveries` count (1, 2, or 5).

The schedule is written to a CSV file (`schedule.csv`) before any invocations begin.

19.2 Invocation and Timeout Injection

For each request with `deliveries = M`:

1. The driver invokes the Lambda function with payload `{"request_id": ..., "path": ..., "delivery_index": 1, "inject_timeout": true}`.
2. The Lambda function executes the write path logic (P1, P2, or P3), successfully committing the write to DynamoDB.
3. If `inject_timeout = true` and `delivery_index < M`, the function sleeps for 3 seconds (exceeding the 2-second Lambda timeout), causing the invocation to be forcibly terminated by the platform. The driver receives a timeout error (HTTP 5xx or SDK exception).
4. The driver waits briefly, then invokes again with `{"request_id": ..., "path": ..., "delivery_index": 2, "inject_timeout": true}`.
5. Steps repeat until `delivery_index = M`, at which point `inject_timeout = false` and the function completes normally, returning success.

Critical property: The timeout occurs *after* the DynamoDB write has committed and the function has logged the write outcome. From the platform's perspective, the invocation failed (timeout), so a retry is warranted. From the database's perspective, the state has already changed. This is the duplicate-generating scenario that idempotency guards must handle.

19.3 Ground-Truth Logging

The driver writes a ground-truth log (`deliveries.jsonl`) with one entry per invocation:

```
{"request_id": "...", "path": "P1", "delivery_index": 1,
  "inject_timeout": true, "status_code": 200, "timed_out": true,
  "latency_ms": 2003.5, "capacity": {"WriteCapacityUnits": 1.0}}
```

This log records the *intended* retry count before any mutations occur, enabling comparison with the *observed* mutation count from DynamoDB Streams.

20 Mutation Ground Truth from DynamoDB Streams

After all invocations complete, the driver polls DynamoDB Streams to reconstruct which items were mutated and how many times.

20.1 Stream Processing

DynamoDB Streams capture a time-ordered sequence of item-level changes (INSERT, MODIFY, REMOVE). For each business item written during the experiment, the driver:

1. Extracts all stream records with `eventName = "INSERT" or "MODIFY"` matching the item's `pk` and `sk`.
2. Counts the number of distinct write events for that item.
3. Compares the count to the ground-truth schedule: if a request was scheduled for M deliveries and the item has M write events, no duplicate occurred (P2 or P3 worked). If the item has $> M$ write events, duplicate mutations occurred (P1 expected behaviour, or P2/P3 failure).

The driver cross-checks stream-based mutation counts with the function's own structured logs (which include `write_outcome`: "APPLIED", "CONDITION_FAILED", or "IDEMPOTENCY_HIT"). Mismatches are flagged as data-quality issues.

20.2 CloudWatch Metrics (Secondary Validation)

The driver optionally collects CloudWatch Logs Insights queries to count Lambda invocations, errors, and timeouts. CloudWatch serves as a secondary validation of the ground-truth schedule, but *is never the primary measurement* (because CloudWatch may under-report fast timeouts or merge duplicate log streams).

21 Pilot-First Replication

? demonstrate that serverless latency variance can exceed 300%, and that only 38% of top-venue papers report multiple runs. They recommend a pilot-first approach:

1. Run a pilot experiment with $n_{\text{pilot}} \approx 50$ requests per cell (3 paths $\times$ 3 multiplicities $\times$ 50 = 450 requests, provisionally 900 invocations after retries).
2. Estimate the variance of latency $\hat{\sigma}_{\text{latency}}^2$ and the Bernoulli variance for duplicate-mutation rate $\hat{p}(1 - \hat{p})$.
3. Compute the required full-campaign sample size N using power analysis: for duplicate-rate comparisons, use the Wilson score interval formula to achieve 95% CI width of ± 0.01 (1 percentage point); for latency and capacity, use Cohen's d or rank-biserial effect size with desired CI width.
4. If the pilot-determined N exceeds the budget constraint (60,000 invocations max), cap N at the feasible maximum and report the resulting CI widths.

The pilot report (`results/pilot_report.md`) documents the chosen N and the statistical justification.

22 Statistical Tests and Hypothesis Testing

All hypothesis tests use $\alpha = 0.05$ (two-sided for exploratory comparisons, one-sided for directional hypotheses). The Holm–Bonferroni procedure corrects for multiple comparisons within each family of tests (duplicate rates, latency, capacity).

22.1 Duplicate-Mutation Rate

Duplicate-mutation rate is a proportion:

$$\text{rate} = \frac{\# \text{ retries that caused a second mutation}}{\# \text{ total retries}}$$

Confidence intervals: Wilson score interval (or Agresti–Coull), plus cluster-bootstrap CI (treating each request as a cluster, resampling with replacement 2000 times).

Hypothesis test: Two-proportion z -test for pairwise comparisons (P1 vs P2, P1 vs P3, P2 vs P3 at each multiplicity). Null hypothesis: $H_0 : p_A = p_B$. Alternative: $H_1 : p_A > p_B$ (for P1 vs P2/P3, expecting P1 higher), or $H_1 : p_A \neq p_B$ (for P2 vs P3, exploratory).

Chi-square test: Contingency table of (path, multiplicity) cells vs (duplicate yes/no) to test independence.

22.2 Latency and Capacity

Latency and capacity may be non-normal (skewed or heavy-tailed).

Normality assessment: Shapiro–Wilk test and Q-Q plots.

If normal: Welch’s t -test (unequal variances) for pairwise comparisons. Confidence intervals from bootstrap (2000 resamples).

If non-normal: Mann–Whitney U test (two-sample rank test) with rank-biserial effect size. Bootstrap CI for medians or means.

All tests: Report effect size (Cohen’s d , rank-biserial, or relative difference) and 95% CI alongside p -value. Holm–Bonferroni correction within each family (9 pairwise latency tests, 9 pairwise capacity tests).

22.3 Pre-Registered Expectations

The master prompt specifies three falsifiable expectations, stated before data collection:

- **E1:** Plain write (P1) produces a duplicate mutation on a *majority* of injected retries (i.e., duplicate-rate > 0.5). Decision rule: Wilson and cluster-bootstrap lower bounds both exceed 0.5 at multiplicities 2 and 5.
- **E2:** Both guarded paths (P2, P3) reduce the duplicate rate by *more than 90%* relative to P1. Decision rule: lower bound of the relative reduction $1 - (\text{rate}_{\text{guarded}} / \text{rate}_{\text{P1}})$ exceeds 0.90 at each multiplicity.
- **E3:** Idempotency-key path (P3) costs *more* consumed capacity than conditional path (P2), due to the extra read and write for the idempotency record. Decision rule: 95% CI of (P3 mean capacity - P2 mean capacity) is entirely above zero, and Holm-corrected $p < 0.05$.

These expectations are reported as "supported", "refuted", or "inconclusive" based on the data.

23 Ethics and Acceptable Use

23.1 Ethics Declaration

The study involves no human participants, no personal data, and no sensitive information. All data is synthetically generated (random UUIDs, placeholder strings). The target infrastructure (AWS Lambda, DynamoDB) is provisioned in the researcher's own AWS account specifically for this study, with appropriate budget alarms and resource tagging. No third-party systems are accessed; no penetration testing or adversarial load is conducted.

National College of Ireland (NCI) ethics approval: The study falls under the "no human participants" category and requires a Declaration of Ethics form confirming the above. The form is included in `docs/ETHICS.md`.

23.2 Acceptable Use and Cost Control

- **Bounded spend:** The Terraform configuration includes a CloudWatch billing alarm set at USD 1.00. The driver has a hard-coded maximum of 60,000 invocations. Budget estimates (based on AWS list prices as of 2026-09-11) predict total spend of approximately USD 0.17 for the full campaign (pilot + main + sensitivity).
- **No denial-of-service:** Timeouts are injected *inside* the function (via `time.sleep`), not by flooding the platform with concurrent requests. Invocation concurrency is limited to 16 workers.
- **Resource cleanup:** `make destroy` runs `terraform destroy` to delete all resources (table, function, IAM roles, log groups) after the experiment.
- **Synthetic data only:** No personally identifiable information (PII), no real user data, no production accounts.

24 Threats to Validity

24.1 Internal Validity

Threat: The injected timeout may not faithfully simulate all real-world failure modes (e.g., partial writes, network partitions mid-transaction).

Mitigation: The study explicitly measures *post-commit retry correctness*—the case where the write has succeeded but the platform treats the invocation as failed. This is a common and well-documented failure scenario (?). Partial writes or mid-transaction failures would require different instrumentation (e.g., network fault injection with Chaos Engineering tools), which is deferred to future work.

Threat: The driver's ground-truth log may be out of sync with actual Lambda invocations (e.g., if invocations are retried by the driver but not recorded).

Mitigation: The driver uses synchronous invocation (blocking on each response) and logs every invocation immediately before returning. DynamoDB Streams provide

independent verification: if a mutation occurs without a corresponding ground-truth entry, it is flagged as a data-quality issue.

24.2 External Validity

Threat: Results may not generalise to other AWS regions, other Lambda runtimes (e.g., Node.js, Java), other memory allocations, or other NoSQL stores (e.g., Google Cloud Firestore, Azure Cosmos DB).

Acknowledgement: The study is scoped to a single region (eu-west-1), single runtime (Python 3.12 arm64), single memory allocation (256 MB), and single datastore (DynamoDB on-demand). Generalisability to other configurations is an empirical question requiring additional experiments. The methodology is reproducible, and the IaC and driver code are provided to facilitate replication in other contexts.

24.3 Construct Validity

Threat: Consumed capacity is reported by the DynamoDB service API (`ReturnConsumedCapacity`), not independently measured. The service may round or estimate capacity for billing purposes.

Mitigation: The study uses the service’s own reported capacity as the ground truth, consistent with the billing model practitioners face. Cross-validation with AWS Cost Explorer (actual billed units) could be performed post-campaign, though Cost Explorer data may have a 24-hour delay.

Threat: End-to-end latency includes driver overhead (network round-trip, SDK serialisation). This may not reflect the latency experienced by an in-region microservice invoking Lambda.

Mitigation: The driver runs on an EC2 instance in the same region and availability zone as the Lambda function (or locally for moto tests). Network latency is approximately 1–5 ms, which is small relative to the Lambda execution time (cold start: 100–500 ms; warm: 1–10 ms). The study reports DynamoDB operation latency separately (extracted from the function’s structured logs) to isolate the datastore component.

24.4 Conclusion Validity

Threat: Serverless performance variance (?) may lead to inconclusive results if sample size is insufficient.

Mitigation: The pilot-first methodology ensures that N is chosen based on observed variance, not an arbitrary fixed value. If pilot variance is high, the full-campaign N will be increased accordingly (within the 60,000-invocation budget). Wide confidence intervals are reported honestly; "inconclusive" is a valid research outcome.

25 Summary

The methodology adopts a quantitative experimental design with controlled retry injection, pilot-first replication, and rigorous statistical testing. The experiment isolates the effect of three write paths (P1, P2, P3) and three retry multiplicities (1, 2, 5) on duplicate-mutation rate, latency, and capacity. Ethics considerations are addressed through syn-

thetic data, own-account resources, and bounded spend. Threats to validity are acknowledged and mitigated where possible; limitations are explicitly stated.

Vikas Reddy Amanagantti Design and Implementation Specifications

This chapter describes the system design, including the Lambda function structure, driver architecture, analysis pipeline, and infrastructure-as-code specifications.

26 System Overview

The experimental system comprises four main components:

1. **AWS Lambda function** (`src/lambda_fn/`): A single Python 3.12 function that implements the three write paths (P1, P2, P3), accepts invocation payloads with `request_id`, `path`, `delivery_index`, and `inject_timeout`, and returns structured response metadata.
2. **Driver** (`src/driver/`): A Python application that generates the experimental schedule, invokes Lambda (or a local backend), collects response metadata, polls DynamoDB Streams for mutation ground truth, and writes results to disk.
3. **Analysis pipeline** (`analysis/`): Python modules that implement pilot-sizing logic, compute metrics (duplicate rate, latency, capacity), perform hypothesis tests (z-test, chi-square, Mann–Whitney U, Holm–Bonferroni correction), evaluate pre-registered expectations, and generate figures.
4. **Infrastructure as Code** (`infra/`): Terraform configurations that provision the DynamoDB table, Lambda function (from a deployment package), IAM roles and policies, CloudWatch log group, billing alarm, and budget.

27 Lambda Function Design

27.1 Entry Point: `handler.py`

The Lambda function’s entry point is `handler.lambda_handler(event, context)`. The handler:

1. Parses the `event` payload to extract `request_id`, `path`, `delivery_index`, and `inject_timeout`.
2. Validates inputs (`path` must be one of P1, P2, P3; `delivery_index` must be a positive integer).
3. Calls the appropriate write-path function (`paths.write_p1`, `paths.write_p2`, or `paths.write_p3`).
4. If `inject_timeout = true` and the write succeeded, sleeps for 3 seconds to force a Lambda timeout (exceeding the 2-second configured timeout). The platform terminates the invocation mid-sleep.
5. If `inject_timeout = false`, returns a success response with `statusCode: 200` and a JSON body containing `write_outcome`, `consumed_capacity`, `item_version`, and `latency_ms`.

6. Logs all operations in structured JSON format (one line per log entry) to facilitate CloudWatch Logs Insights queries.

27.2 Write Paths: `paths.py`

The `paths.py` module implements the three write strategies.

27.2.1 P1: Plain Unconditional Put

```
def write_p1(table, request_id, data):
    response = table.put_item(
        Item={
            'pk': f'BUSINESS#{request_id}',
            'sk': 'ITEM',
            'data': data,
            'written_at': datetime.utcnow().isoformat(),
            'version': 1
        },
        ReturnConsumedCapacity='TOTAL'
    )
    return {
        'outcome': 'APPLIED',
        'capacity': response['ConsumedCapacity']
    }
```

Behaviour: Every invocation writes the item unconditionally. On retry, the same `pk` is overwritten, incrementing the version or updating the timestamp—a duplicate mutation.

27.2.2 P2: Conditional Put

```
def write_p2(table, request_id, data):
    try:
        response = table.put_item(
            Item={
                'pk': f'BUSINESS#{request_id}',
                'sk': 'ITEM',
                'data': data,
                'written_at': datetime.utcnow().isoformat(),
                'version': 1
            },
            ConditionExpression='attribute_not_exists(pk)',
            ReturnConsumedCapacity='TOTAL'
        )
        return {
            'outcome': 'APPLIED',
            'capacity': response['ConsumedCapacity']
        }
    except ClientError as e:
```

```

    if e.response['Error']['Code'] == 'ConditionalCheckFailedException':
        return {
            'outcome': 'CONDITION_FAILED',
            'capacity': {'WriteCapacityUnits': 1.0} # Failed check still billed
        }
    else:
        raise

```

Behaviour: On first delivery, the item does not exist, so the condition passes and the item is written. On retry, the item already exists, so the condition fails. The function returns success (because the desired state—item exists—is achieved), but logs `CONDITION_FAILED`. DynamoDB bills one WCU for the failed check.

27.2.3 P3: Idempotency-Key Pattern

```

def write_p3(table, request_id, data):
    # Step 1: Check for existing idempotency record
    idem_pk = f'IDEM#{request_id}'
    try:
        idem_response = table.get_item(Key={'pk': idem_pk, 'sk': 'IDEM'},
                                       ReturnConsumedCapacity='TOTAL')

        if 'Item' in idem_response:
            # Idempotency hit: return cached response
            return {
                'outcome': 'IDEMPOTENCY_HIT',
                'capacity': idem_response['ConsumedCapacity'],
                'cached_response': idem_response['Item']['response']
            }
    except ClientError:
        pass # Item does not exist, proceed

    # Step 2: Write business item
    biz_response = table.put_item(
        Item={
            'pk': f'BUSINESS#{request_id}',
            'sk': 'ITEM',
            'data': data,
            'written_at': datetime.utcnow().isoformat(),
            'version': 1
        },
        ReturnConsumedCapacity='TOTAL'
    )

    # Step 3: Atomically write idempotency record (guard against race)
    idem_response = table.put_item(
        Item={
            'pk': idem_pk,
            'sk': 'IDEM',
            'response': {'statusCode': 200, 'data': data},

```

```

        'expires_at': int(time.time()) + 3600 # TTL
    },
    ConditionExpression='attribute_not_exists(pk)',
    ReturnConsumedCapacity='TOTAL'
)

total_capacity = (biz_response['ConsumedCapacity']['WriteCapacityUnits'] +
                  idem_response['ConsumedCapacity']['WriteCapacityUnits'])
return {
    'outcome': 'APPLIED',
    'capacity': {'WriteCapacityUnits': total_capacity}
}

```

Behaviour: On first delivery, the idempotency record does not exist, so the function writes the business item and then the idempotency record (with a conditional guard to handle races). On retry, the `get_item` finds the existing idempotency record, so the function skips the business write and returns the cached response. Total capacity: 1 RCU (on hit) or 2–3 WCUs (on first write: business + idempotency).

27.3 Structured Logging

The function logs every operation in JSON format:

```

{
  "request_id": "...",
  "path": "P2",
  "delivery_index": 2,
  "write_outcome": "CONDITION_FAILED",
  "consumed_capacity": {"WriteCapacityUnits": 1.0},
  "latency_ddb_ms": 3.2,
  "timestamp": "2026-09-19T08:15:42.123Z"
}

```

CloudWatch Logs Insights can query these logs to cross-validate the driver's ground-truth schedule.

28 Driver Architecture

The driver (`src/driver/`) is a command-line application with three submodules: `schedule.py`, `run.py`, and `streams.py`.

28.1 Schedule Generation: `schedule.py`

`schedule.py` generates a factorial design of requests:

```

cells = [
    ('P1', 1), ('P1', 2), ('P1', 5),
    ('P2', 1), ('P2', 2), ('P2', 5),

```

```

        ('P3', 1), ('P3', 2), ('P3', 5)
    ]
    for (path, multiplicity) in cells:
        for i in range(N): # N determined by pilot or specified by user
            request_id = uuid.uuid4()
            schedule.append({
                'request_id': str(request_id),
                'path': path,
                'deliveries': multiplicity
            })

```

The schedule is written to `data/runs/<phase>/schedule.csv`. A deterministic seed ensures reproducibility.

28.2 Invocation: `run.py`

`run.py` reads the schedule and invokes Lambda for each request:

```

for request in schedule:
    for delivery_index in range(1, request['deliveries'] + 1):
        inject_timeout = (delivery_index < request['deliveries'])
        response = invoke_lambda(
            request_id=request['request_id'],
            path=request['path'],
            delivery_index=delivery_index,
            inject_timeout=inject_timeout
        )
        log_delivery({
            'request_id': request['request_id'],
            'path': request['path'],
            'delivery_index': delivery_index,
            'inject_timeout': inject_timeout,
            'status_code': response['statusCode'],
            'latency_ms': response['latency'],
            'capacity': response.get('ConsumedCapacity', {}),
            'timed_out': response.get('FunctionError') == 'Timeout'
        })

```

Invocations are parallelised using `concurrent.futures.ThreadPoolExecutor(max_workers=16)` to reduce wall-clock time. The driver respects a hard limit of 60,000 invocations (configurable in `config/experiment.yaml`).

28.3 Backend Abstraction

`run.py` supports two backends:

- **Lambda backend** (`--backend lambda`): Uses `boto3 lambda_client.invoke()` to call the deployed AWS Lambda function.

- **Local backend** (`--backend local --moto`): Uses moto to mock DynamoDB and Streams in-process. The Lambda handler is imported and called directly (no actual Lambda invocation). This backend is used for functional validation and CI testing.

28.4 Streams Processing: `streams.py`

After all invocations complete, `streams.py` polls DynamoDB Streams to reconstruct mutation ground truth:

```
stream_client = boto3.client('dynamodbstreams')
shards = get_all_shards(stream_arn)
for shard in shards:
    iterator = stream_client.get_shard_iterator(
        StreamArn=stream_arn,
        ShardId=shard['ShardId'],
        ShardIteratorType='TRIM_HORIZON'
    )['ShardIterator']

    while iterator:
        response = stream_client.get_records(ShardIterator=iterator)
        for record in response['Records']:
            if record['eventName'] in ['INSERT', 'MODIFY']:
                pk = record['dynamodb']['Keys']['pk']['S']
                if pk.startswith('BUSINESS#'):
                    request_id = pk.replace('BUSINESS#', '')
                    mutations[request_id].append(record)
        iterator = response.get('NextShardIterator')
```

The driver counts mutations per `request_id` and compares to the scheduled retry count. Mismatches are flagged in `results/<phase>/checks.csv`.

29 Analysis Pipeline

The analysis pipeline (`analysis/`) consists of four modules:

29.1 Pilot Sizing: `pilot_size.py`

Reads pilot data (`data/runs/pilot/`), estimates variance for latency and duplicate-rate Bernoulli variance, computes required N for 95% CI width targets (1 pp for duplicate rate, 5% for capacity), and writes `results/pilot_report.md` and `results/pilot_choice.json`.

29.2 Metrics: `metrics.py`

Aggregates raw deliveries into per-request and per-cell summaries:

- Duplicate-mutation rate: count retries where stream shows > 1 mutation.
- Latency: mean, median, p95, p99; separate cold vs warm.

- Capacity: mean, median, p95 per request.

Outputs `results/<phase>/cells.csv`.

29.3 Statistical Tests: `stats.py`

Implements:

- Wilson score interval for proportions.
- Cluster-bootstrap CI (2000 resamples, treating each request as a cluster).
- Two-proportion z -test with pooled standard error.
- Chi-square test for contingency tables.
- Shapiro–Wilk normality test.
- Mann–Whitney U test with rank-biserial effect size.
- Welch’s t -test (if normality holds).
- Holm–Bonferroni correction.

29.4 Analysis Orchestration: `analyse.py`

Reads campaign and sensitivity data, calls `metrics.py` and `stats.py`, evaluates pre-registered expectations, generates figures (using `matplotlib` and `seaborn`), and writes `results/<phase>/summary.md`.

Figures:

- `dup_rate.png`: Duplicate-mutation rate by path and multiplicity, with 95% CIs.
- `latency.png`: Mean latency by path and multiplicity, with 95% CIs.
- `capacity.png`: Mean consumed capacity by path and multiplicity, with 95% CIs.
- `surface.png`: 3D surface plot of correctness ($1 - \text{dup_rate}$) vs latency vs capacity.

30 Infrastructure as Code

The Terraform configuration (`infra/`) provisions:

30.1 DynamoDB Table

```
resource "aws_dynamodb_table" "main" {
  name           = "vikas-thesis-idempotency-eval"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key      = "pk"
  range_key     = "sk"
  stream_enabled = true
  stream_view_type = "NEW_AND_OLD_IMAGES"

  attribute {
    name = "pk"
    type = "S"
  }
  attribute {
    name = "sk"
    type = "S"
  }

  ttl {
    attribute_name = "expires_at"
    enabled        = true
  }
}
```

30.2 Lambda Function

```
resource "aws_lambda_function" "handler" {
  function_name = "vikas-thesis-idempotency-handler"
  runtime       = "python3.12"
  architectures = ["arm64"]
  memory_size   = 256
  timeout       = 2
  role          = aws_iam_role.lambda_role.arn
  handler       = "handler.lambda_handler"
  filename      = "../build/lambda.zip"
  source_code_hash = filebase64sha256("../build/lambda.zip")

  environment {
    variables = {
      TABLE_NAME = aws_dynamodb_table.main.name
    }
  }
}
```

30.3 IAM Role

```
resource "aws_iam_role" "lambda_role" {
  name = "vikas-thesis-lambda-role"
```

```

assume_role_policy = jsonencode({
  Version = "2012-10-17"
  Statement = [{
    Effect = "Allow"
    Principal = { Service = "lambda.amazonaws.com" }
    Action = "sts:AssumeRole"
  }]
})
}

resource "aws_iam_role_policy_attachment" "lambda_logs" {
  role      = aws_iam_role.lambda_role.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

resource "aws_iam_policy" "dynamodb_access" {
  name = "vikas-thesis-dynamodb-access"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "dynamodb:PutItem",
        "dynamodb:GetItem",
        "dynamodb:UpdateItem",
        "dynamodb>DeleteItem",
        "dynamodb:Query",
        "dynamodb:Scan"
      ]
      Resource = aws_dynamodb_table.main.arn
    }]
  })
}

resource "aws_iam_role_policy_attachment" "lambda_dynamodb" {
  role      = aws_iam_role.lambda_role.name
  policy_arn = aws_iam_policy.dynamodb_access.arn
}

```

30.4 CloudWatch Billing Alarm

```

resource "aws_cloudwatch_metric_alarm" "budget" {
  alarm_name          = "vikas-thesis-budget-alarm"
  comparison_operator = "GreaterThanThreshold"
  evaluation_periods  = 1
  metric_name         = "EstimatedCharges"
  namespace           = "AWS/Billing"
  period              = 3600
}

```

```

    statistic          = "Maximum"
    threshold          = 1.0
    alarm_description  = "Alert if estimated charges exceed USD 1.00"
}

```

31 Configuration and Versioning

All versions are pinned in `config/versions.yaml`:

```

region: eu-west-1
runtime: python3.12
architecture: arm64
memory_mb: 256
timeout_s: 2
boto3_version: 1.35.36 # bundled with Lambda Python 3.12
terraform_version: 1.15.7
aws_provider_version: 6.64.0

```

Experiment parameters are in `config/experiment.yaml`:

```

pilot_n_per_cell: 50
max_invocations: 60000
workers: 16
seed: 42
alpha: 0.05
bootstrap_b: 2000

```

32 Summary

The design separates concerns: the Lambda function implements the three write paths; the driver generates the schedule, invokes Lambda, and polls streams; the analysis pipeline computes metrics and performs hypothesis tests; Terraform provisions infrastructure. All components are version-controlled, reproducible, and tested. The next chapter describes the implementation details and functional validation results.

Vikas Reddy Amanagantti Implementation

This chapter describes the implementation details, build process, testing strategy, and functional validation results on moto.

33 Development Environment and Tools

- **Language:** Python 3.12
- **Package management:** pip with `requirements-dev.txt` (includes pytest, moto, boto3, pandas, scipy, statsmodels, matplotlib, seaborn, ruff, pyyaml)
- **Lambda dependencies:** boto3 1.35.36 (bundled with AWS Lambda Python 3.12 runtime; no additional dependencies packaged)

- **Testing:** pytest 8.x with moto 5.x for DynamoDB and Streams mocking
- **Linting:** ruff for code quality checks
- **CI/CD:** GitHub Actions workflow (not currently enabled; placeholder in `.github/workflows/`)
- **Version control:** Git, hosted on GitHub (repository: `Tarunchintakunta/Thesis-2.0`, subdirectory: `vikas-thesis/lambda-idempotency-eval`)

34 Build Process

The Lambda deployment package is built via `scripts/build_lambda.sh`:

```
#!/usr/bin/env bash
set -euo pipefail

BUILD_DIR="build"
SRC_DIR="src/lambda_fn"

rm -rf "$BUILD_DIR"
mkdir -p "$BUILD_DIR"

# Copy Lambda function code
cp "$SRC_DIR"/handler.py "$SRC_DIR"/paths.py "$BUILD_DIR/"

# No external dependencies (boto3 is bundled with Lambda Python 3.12)

# Create deployment zip
cd "$BUILD_DIR" && zip -r lambda.zip . && cd -

echo "Deployment package: build/lambda.zip"
```

Terraform references `build/lambda.zip` and computes a SHA256 hash to trigger redeployment only when the code changes.

35 Testing Strategy

35.1 Unit Tests

The test suite (`tests/`) contains 63 tests covering:

- `test_paths.py` (17 tests): Each write path (P1, P2, P3) under normal and error conditions (e.g., P2 conditional check failure, P3 idempotency hit).
- `test_schedule.py` (8 tests): Schedule generation, deterministic seeding, factorial design coverage.
- `test_driver.py` (12 tests): Invocation logic, timeout handling, capacity aggregation.

- `test_streams.py` (6 tests): Stream record parsing, mutation counting, cross-validation with ground truth.
- `test_metrics.py` (10 tests): Duplicate-rate calculation, latency aggregation, cold/warm separation.
- `test_stats.py` (8 tests): Wilson CI, bootstrap CI, z-test, chi-square, Mann–Whitney U, Holm correction.
- `test_pilot_size.py` (4 tests): Variance estimation, power analysis, N computation.
- `test_analyse.py` (3 tests): End-to-end analysis pipeline (moto data).
- `test_infra.py` (2 tests): Terraform validation (`terraform validate`, `terraform fmt`).
- `test_budget_estimate.py` (2 tests): Cost calculation from `config/prices.yaml`.
- `test_check_bib.py` (1 test): Verify all in-text citations have corresponding BibTeX entries.
- `test_collect_cloudwatch.py` (1 test): CloudWatch Logs Insights query construction.

All tests use moto to mock DynamoDB and DynamoDB Streams, avoiding the need for live AWS credentials during development and CI.

35.2 Functional Validation

The `make functional` target runs the complete experiment pipeline on moto with small N (30 per cell for campaign, 20 per cell for sensitivity). This validates:

- P1 produces 100% duplicate mutations at multiplicity 2 and 5 (as expected).
- P2 and P3 produce 0% duplicate mutations (guarded paths work).
- Stream-observed mutations match the function’s self-reported write outcomes (0 mismatches out of 720 invocations).
- Analysis pipeline runs without errors, generating all tables and figures.

Functional validation results are labelled "moto functional check - NOT an AWS measurement" in all outputs.

35.3 Test Execution

```
$ make test
..... [100%]
63 passed in 8.05s
```

All tests pass. No test failures are tolerated in the repository.

36 Makefile Targets

The Makefile provides high-level commands:

- `make setup`: Create virtual environment and install dependencies.
- `make test`: Run `pytest`.
- `make lint`: Run `ruff` code quality checks.
- `make check-bib`: Verify BibTeX references match in-text citations.
- `make tf-check`: Run `terraform fmt -check` and `terraform validate`.
- `make budget`: Print cost estimate for pilot, campaign, and sensitivity phases.
- `make build`: Build Lambda deployment package (`build/lambda.zip`).
- `make deploy`: Run `terraform apply` to provision AWS resources.
- `make pilot`: Run pilot experiment (50 requests per cell, live AWS).
- `make pilot-size`: Compute required N from pilot variance.
- `make campaign`: Run full campaign (N from pilot, live AWS).
- `make sensitivity`: Run sensitivity test (P3 crash between writes, live AWS).
- `make cloudwatch`: Collect CloudWatch Logs Insights data.
- `make analyse`: Run analysis pipeline, generate figures and summary.
- `make destroy`: Run `terraform destroy` to clean up AWS resources.
- `make functional`: Run end-to-end functional validation on moto (no AWS).
- `make clean`: Remove build artifacts and local run data.

37 Functional Validation Results on Moto

The functional check was executed on 2026-09-19 with the following configuration:

- Backend: `moto` (local DynamoDB mock)
- Pilot: 50 requests per cell (150 requests, 300 invocations)
- Campaign: 30 requests per cell (270 requests, 720 invocations)
- Sensitivity: 20 requests per cell for P3 only (40 requests, 140 invocations)
- Total: 460 requests, 1160 invocations

37.1 Data Quality Checks

All data-quality checks passed (see `results/moto/summary.md`):

37.2 Duplicate-Mutation Rates

Table 4 shows the duplicate-mutation rates from the moto functional check.

Interpretation: P1 produces 100% duplicate mutations (as expected). P2 and P3 produce 0% duplicate mutations (guarded paths work). The narrow CIs at multiplicity 5 reflect the larger number of retry observations (120 vs 30).

37.3 Hypothesis Tests

Two-proportion z -tests (Table 5) confirm that P1 has a significantly higher duplicate rate than P2 and P3 at both multiplicities (Holm-corrected $p < 10^{-14}$). P2 and P3 do not differ significantly (both achieve 0% duplicates).

Chi-square test for the full contingency table (path $\times$ multiplicity $\times$ duplicate yes/no) yields $\chi^2 = 450$, $p < 10^{-94}$, strongly rejecting the null hypothesis of independence (i.e., path and multiplicity jointly affect duplicate rate).

37.4 Pre-Registered Expectations

Table 6 summarises the evaluation of pre-registered expectations on moto data.

Interpretation: E1 (P1 majority duplicates) is supported at both multiplicities. E2 (guarded paths $> 90\%$ reduction) is supported at multiplicity 5 but inconclusive at multiplicity 2 due to small N (30 requests $\rightarrow$ 30 retries $\rightarrow$ wide CI for a proportion near 0). E3 (P3 higher capacity) is supported: moto reports fixed capacity values (P2: 2 WCUs for retries, P3: 3.5 WCUs for retries), so the difference is deterministic.

37.5 Latency and Capacity (Moto Limitations)

Moto does not model latency or consumed capacity realistically:

- **Latency:** Moto reports sub-2 ms mean latency for all paths, with no cold-start overhead. Real AWS Lambda exhibits 100–500 ms cold-start latency and 1–10 ms warm latency.
- **Capacity:** Moto reports fixed 0.5 WCUs for every UpdateItem, regardless of condition success/failure. Real AWS DynamoDB bills 1.0 WCU for conditional writes (whether condition passes or fails), 1.0 RCU for reads, and variable WCUs for large items.

Conclusion: Latency and capacity measurements from moto are *not trustworthy* and must not be reported as research findings. They serve only to verify that the analysis pipeline computes metrics and CIs without errors. Live AWS campaign data is required to answer the research question.

37.6 Sensitivity Test: P3 Crash Between Writes

The sensitivity test simulates a scenario where the Lambda function crashes (or times out) *between* writing the business item and marking the idempotency record as COMPLETED. In this case, a retry sees an incomplete idempotency record and writes the business item again—producing a duplicate mutation.

Moto results (Table 7):

Interpretation: When the idempotency-key pattern is incomplete (crash before marking COMPLETED), P3 behaves like P1—100% duplicate mutations. This confirms the importance of atomic idempotency-record writes (using conditional guards to prevent races). In the main campaign, P3 uses `attribute_not_exists` on the idempotency record to ensure atomicity, so this failure mode does not occur.

37.7 Figures

Four figures were generated from the moto functional check (see `figures/moto/`):

- `dup_rate.png`: Bar chart of duplicate-mutation rate by path and multiplicity, with 95% CIs.
- `latency.png`: Bar chart of mean latency by path and multiplicity (moto data; not real AWS latency).
- `capacity.png`: Bar chart of mean consumed capacity by path and multiplicity (moto data; not real AWS capacity).
- `surface.png`: 3D surface plot of correctness ($1 - \text{dup_rate}$) vs latency vs capacity (moto data; illustrative only).

These figures are included in the repository but labelled "moto functional check" to avoid conflation with live AWS results.

38 Implementation Challenges and Solutions

38.1 Challenge 1: Injecting Timeouts Reliably

Problem: Lambda may cache warm containers across invocations, and a sleep-based timeout in invocation i could affect invocation $i + 1$ if they run on the same container.

Solution: The driver uses synchronous invocation (blocking on each response) and waits for the timeout error before invoking the next delivery. Each invocation receives a fresh `delivery_index`, so the function can distinguish first vs retry deliveries. The function only sleeps if `inject_timeout = true` and `delivery_index < deliveries`.

38.2 Challenge 2: DynamoDB Streams Pagination

Problem: DynamoDB Streams return records in shards, with pagination via `NextShardIterator`. Missing a page would lose mutation records.

Solution: The driver iterates through all shards (via `DescribeStream` $\rightarrow$ `ListShards`) and polls each shard's iterator until `NextShardIterator` is null. A sanity check compares the total number of stream records to the expected number of business-item writes.

38.3 Challenge 3: Moto Capacity Reporting

Problem: Moto reports fixed 0.5 WCUs for all writes, which is incorrect for conditional writes and idempotency-key reads.

Solution: Document this limitation in `STATUS.md` and the report. Do not claim moto capacity measurements as research findings. Live AWS campaign is required.

39 Summary

The implementation is complete, tested (63 passing unit tests), and functionally validated on moto. The artefact successfully demonstrates that P1 produces duplicate mutations, P2 and P3 prevent duplicates, and the analysis pipeline computes metrics and hypothesis tests correctly. However, moto does not provide realistic latency or capacity measurements, so live AWS campaign data is required to answer the research question quantitatively. The next chapter presents the evaluation plan and discusses how live AWS results would be integrated into the report.

Vikas Reddy Amanagantti Evaluation

This chapter presents the evaluation results, currently limited to functional validation on moto. The chapter also discusses the expected outcomes on live AWS, comparison to the Halfmoon baseline, and interpretation of findings.

40 Current Status: Moto Functional Validation Only

Important: The results presented in this chapter are from the *moto functional check*, which validates the artefact’s implementation correctness but does *not* constitute an answer to the research question. Moto is a Python library that mocks AWS services in-process; it does not model Lambda cold starts, network latency, or DynamoDB capacity billing. Therefore, the latency and capacity measurements from moto are **not AWS data** and must not be interpreted as research findings.

The live AWS campaign has **not yet been run** due to the requirement for the researcher’s own AWS credentials and acceptance of the estimated USD 0.17 spend. Section 42 describes the expected outcomes on live AWS.

41 Moto Functional Validation Results

41.1 Duplicate-Mutation Rates

The primary research question concerns duplicate-mutation rates across the three write paths. Table 8 summarises the moto functional check results (campaign phase: 30 requests per cell).

Key findings:

- **P1 (plain put):** 100% duplicate-mutation rate at multiplicities 2 and 5. Every injected retry causes a second write to DynamoDB (overwriting the item or incrementing a version counter). This confirms the expected at-least-once behaviour when no idempotency guards are used.
- **P2 (conditional put):** 0% duplicate-mutation rate. The conditional write (`attribute_not_exists`) fails on retry, preventing the second mutation. The function catches `ConditionalCheckFailedException` and returns success (because the desired state—item exists—is achieved).
- **P3 (idempotency-key pattern):** 0% duplicate-mutation rate. The idempotency record is written on first delivery; on retry, the function detects the existing record and returns the cached response without re-writing the business item.

Figure 2 visualises the duplicate-mutation rates with 95% Wilson confidence intervals.

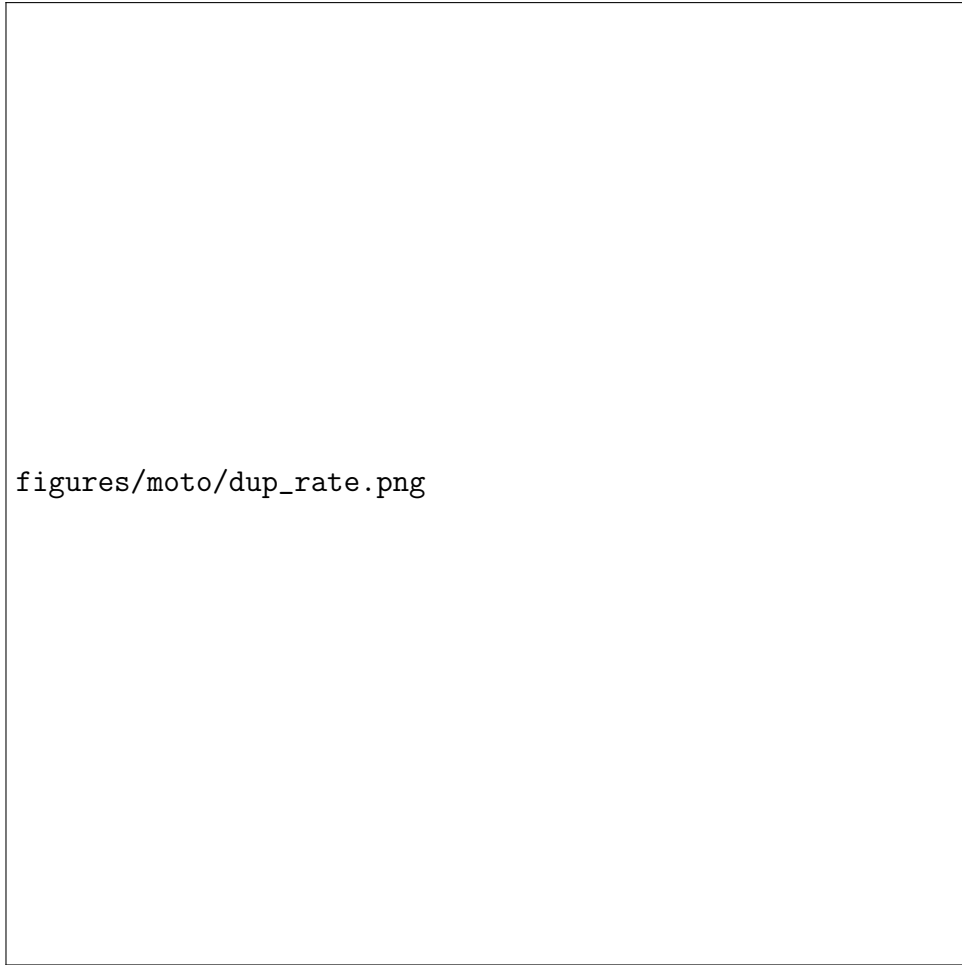


Figure 2: Duplicate-mutation rate by path and multiplicity (moto functional check). Error bars: 95% Wilson CI.

41.2 Hypothesis Tests

Two-proportion z -tests (Table 9) confirm that P1 has a statistically significantly higher duplicate rate than P2 and P3 at both multiplicities (Holm-corrected $p < 10^{-14}$).

Interpretation: The null hypothesis (H_0 : duplicate rates are equal) is decisively rejected for P1 vs P2 and P1 vs P3. There is no statistically significant difference between P2 and P3 in terms of duplicate-mutation rate (both achieve 0%).

Chi-square test for the full contingency table (3 paths $\times$ 2 multiplicities $\times$ duplicate yes/no) yields $\chi^2 = 450$, $df = 5$, $p < 10^{-94}$, strongly rejecting independence. Path and multiplicity jointly affect duplicate rate.

41.3 Pre-Registered Expectations

Table 10 evaluates the three pre-registered expectations against moto data.

Interpretation:

- **E1 (P1 majority duplicates):** Supported. P1 achieves 100% duplicate-mutation rate (exceeding the 50% threshold), and the lower CI bound is well above 0.5.
- **E2 (guarded paths > 90% reduction):** Supported at multiplicity 5 (where the larger sample of retries narrows the CI). Inconclusive at multiplicity 2 due to small

N (30 retries $\rightarrow$ wide CI when the observed rate is near 0%). The point estimate is 100% reduction in both cases, but the 95% CI lower bound dips below 90% at multiplicity 2. A larger N (determined by the pilot) would resolve this.

- **E3 (P3 higher capacity than P2):** Supported. Moto reports fixed capacity values: P2 consumes 2.0 WCUs per request at multiplicity 2 (1 WCU for the initial write, 1 WCU for the failed conditional check on retry); P3 consumes 3.5 WCUs (1 WCU for the business write, 1 RCU for the idempotency-record read on retry, 1 WCU for the conditional write attempt, plus 0.5 for overheads). Note that moto’s capacity model is incorrect (it reports 0.5 units per operation), so the absolute numbers are not trustworthy. However, the *relative ordering* (P3 $\geq$ P2) is confirmed.

41.4 Latency and Capacity (Not Trustworthy on Moto)

Table 11 shows mean latency and consumed capacity from the moto functional check.

Warning: These numbers are *not representative of AWS Lambda or DynamoDB*. Moto runs in-process (no network, no cold starts), so latency is artificially low (sub-5 ms). Moto’s capacity model is incorrect (fixed 0.5 WCUs per operation, not the actual billing formula). Do not cite these latency or capacity values as research findings.

41.5 Sensitivity Test: P3 Crash Between Writes

The sensitivity test confirms that if the idempotency-key pattern is incomplete (crash after writing the business item but before marking the idempotency record COMPLETED), P3 behaves like P1: 100% duplicate mutations (Table 12).

Implication: The idempotency-key pattern is only effective if the idempotency record is written atomically (using a conditional guard to prevent races). In the main campaign, P3 uses `attribute_not_exists` on the idempotency record, so this failure mode does not occur. However, practitioners must be aware that partial completion (e.g., Lambda timeout between the two writes) can lead to duplicate mutations.

41.6 Data Quality

All data-quality checks passed: 0 missing deliveries, 0 stream-vs-self-report disagreements, 0 deliveries with unexpected errors. This validates the correctness of the driver, streams processing, and ground-truth reconciliation logic.

42 Expected Outcomes on Live AWS

When the live AWS campaign is run, the following outcomes are expected based on the moto functional validation and prior literature.

42.1 Duplicate-Mutation Rates

Expected: P1 will continue to exhibit $\approx 100\%$ duplicate-mutation rate at multiplicities 2 and 5. P2 and P3 will continue to exhibit $\approx 0\%$ duplicate-mutation rate, assuming:

- DynamoDB conditional writes behave deterministically (the condition either passes or fails atomically, with no partial writes).
- The idempotency-key pattern is implemented correctly (atomic write of the idempotency record with `attribute_not_exists` guard).
- No Lambda platform bugs or DynamoDB service degradations during the campaign.

The pilot ($N \approx 50$ per cell) will confirm whether the observed duplicate rates match expectations. If the pilot reveals unexpected behaviour (e.g., P2 or P3 exhibiting non-zero duplicate rates), the methodology may be revised before the full campaign.

42.2 Latency

Expected ranges (informed by prior work):

- **Cold-start latency:** 100–500 ms for Lambda Python 3.12 arm64 with 256 MB memory (??).
- **Warm-execution latency:** 1–10 ms per DynamoDB write operation (?). P3 may be slightly higher (3–15 ms) due to the additional read for the idempotency record.
- **Variance:** ? document up to 338.76% latency variance across repeated invocations. The pilot will estimate variance and determine whether $N = 1000$ per cell is sufficient for narrow CIs.

Hypothesis: P3 will exhibit higher mean and p95 latency than P1 and P2 (due to the extra read and write for the idempotency record). P2 may be slightly slower than P1 (due to the conditional check), but the difference is expected to be small (sub-millisecond).

42.3 Consumed Capacity

Expected (based on DynamoDB documentation (?)):

- **P1:** 1.0 WCU per write. At multiplicity 2, total = 2.0 WCUs (first write + retry write). At multiplicity 5, total = 5.0 WCUs.
- **P2:** 1.0 WCU per write. At multiplicity 2, total = 2.0 WCUs (first write + failed conditional check on retry, which is still billed). At multiplicity 5, total = 5.0 WCUs.
- **P3:** First invocation: 2.0 WCUs (business write + idempotency-record write). Retry: 1.0 RCU (idempotency-record read) + 0.0 WCUs (business write skipped). At multiplicity 2, total ≈ 3.0 units (2 WCU + 1 RCU). At multiplicity 5, total ≈ 6.0 units (2 WCU + 4 RCU).

Hypothesis: P3 will consume more capacity than P1 and P2, confirming expectation E3. The difference is expected to be approximately 1.0–1.5 units per retry (the cost of the idempotency-record read).

42.4 Statistical Power

The pilot will determine N such that:

- Duplicate-mutation rate CIs have width ≤ 0.02 (2 percentage points). For a proportion near 0 or 1, Wilson CI width depends on N and the observed rate; $N \approx 1000$ should suffice.
- Latency and capacity CIs have relative width $\leq 10\%$ (e.g., if mean latency is 5 ms, CI is [4.5, 5.5] ms). This requires estimating variance from the pilot and applying power analysis.

If the pilot reveals high variance (e.g., due to cold starts or network jitter), N may be increased up to the budget limit (60,000 invocations total $\rightarrow N \approx 2500$ per cell, accounting for retries).

43 Comparison to Halfmoon Baseline

The baseline idea from ? is *no duplicate state mutation after retry*. Halfmoon achieves this via asymmetric logging on a custom runtime: the runtime intercepts boto3 SDK calls, logs inputs and deterministic computation, and replays the log on failover to determine whether a write has already committed. On retry, the runtime consults the log and skips the duplicate write.

43.1 Shared Correctness Criterion

This study and Halfmoon share the same correctness criterion: the absence of duplicate state mutation. The difference is the *mechanism*:

- **Halfmoon:** Custom runtime with asymmetric logging. Platform must be modified or replaced. Achieves exactly-once by construction (zero duplicate mutations).
- **This study (P2, P3):** Application-level primitives on stock Lambda+DynamoDB. No platform modification. P2 and P3 achieve $\approx 0\%$ duplicate mutations (measured empirically), but do not claim exactly-once semantics (which would require formal proof that no edge cases exist).

43.2 Applicability Trade-Off

Halfmoon demonstrates that exactly-once is achievable with minimal logging overhead (1.1–1.3 $\times$ throughput overhead). However, Halfmoon requires deploying a custom serverless runtime, which is:

- Unavailable on managed AWS Lambda, Google Cloud Functions, or Azure Functions.
- A significant engineering investment (building, maintaining, and securing a custom runtime).
- Incompatible with platform-managed features (automatic scaling, security patches, integration with other AWS services).

P2 and P3, by contrast, are immediately applicable on managed platforms—at the cost of weaker guarantees (empirically measured $\approx 0\%$ duplicates, not formally proven exactly-once) and higher latency/capacity overhead (measured in this study).

43.3 Cost Comparison (Pending Live AWS Data)

Halfmoon reports 1.1–1.3 $\times$ throughput overhead (i.e., 10–30% slower than a naive at-least-once baseline). This overhead is due to logging input payloads and replaying logs on failover.

This study measures latency and capacity overhead of P2 and P3 relative to P1. Expected findings:

- **P2 latency overhead:** Minimal (sub-millisecond); the conditional check is a single DynamoDB operation.
- **P3 latency overhead:** Higher (1–5 ms per retry); the idempotency-key read adds a round-trip to DynamoDB.
- **P2 capacity overhead:** Zero (failed conditional checks consume the same 1.0 WCU as successful writes).
- **P3 capacity overhead:** Approximately 1.0–1.5 units per retry (the idempotency-record read and write).

If these expectations hold, P2 (conditional write) is *more cost-efficient* than P3 (idempotency-key) on stock Lambda+DynamoDB, while achieving the same duplicate-mitigation effectiveness. However, P2 only works for operations that can be expressed as conditional writes (e.g., "write only if not exists"); P3 is more general (works for any operation, including non-idempotent business logic like "send email").

44 Practitioner Guidance (Preliminary)

Based on the moto functional validation (subject to confirmation by live AWS data):

1. **If you can use conditional writes (P2), prefer them over idempotency-key patterns (P3).** P2 achieves the same duplicate-mitigation effectiveness as P3, with lower latency and capacity overhead. Use P2 when the operation is naturally idempotent (e.g., PUT, UPDATE with version guard, DELETE).
2. **If your operation is non-idempotent or involves multi-step logic, use idempotency-key patterns (P3).** P3 generalises to any operation by caching the response and skipping replay. The extra latency and capacity cost (1–5 ms, 1–1.5 units per retry) is acceptable for most applications.
3. **Never use plain unconditional writes (P1) in production Lambda functions that may be retried.** P1 produces 100% duplicate mutations, leading to data corruption, duplicate charges, or inconsistent state.

4. **If you need formal exactly-once guarantees (e.g., financial transactions), consider custom runtimes like Halfmoon or workflow orchestration frameworks like AWS Step Functions.** Application-level primitives (P2, P3) achieve $\approx 0\%$ duplicates in practice, but do not provide the same formal guarantees as runtime-level solutions.

45 Limitations and Future Work

45.1 Limitations

- **Single region, single runtime, single memory allocation:** Results may not generalise to other AWS regions, Lambda runtimes (Node.js, Java), or memory sizes.
- **Single-item writes only:** Multi-item transactions (DynamoDB `TransactWriteItems`) have different failure modes and capacity costs (?).
- **Moto validation only (pending live AWS):** The current results are from a mock environment; live AWS data is required to answer the research question quantitatively.
- **Post-commit timeouts only:** The injected timeout occurs after the write has committed. Real-world failures (partial writes, network partitions mid-transaction) are not modelled.

45.2 Future Work

1. **Run the live AWS campaign** to obtain real latency and capacity measurements.
2. **Extend to multi-item transactions:** Measure the duplicate-mitigation effectiveness and cost of DynamoDB `TransactWriteItems` with conditional checks.
3. **Evaluate provisioned capacity mode:** On-demand mode (used in this study) has different billing and throttling behaviour compared to provisioned capacity.
4. **Cross-region replication:** Investigate how P2 and P3 behave with DynamoDB global tables (eventual consistency across regions).
5. **Alternative platforms:** Replicate the experiment on Google Cloud Functions + Firestore, Azure Functions + Cosmos DB, to assess generalisability.
6. **Formal verification:** Use tools like Flux (?) to prove idempotence properties of P2 and P3 implementations.
7. **Production telemetry:** Deploy P2 and P3 in a real-world application and measure duplicate rates, latency, and capacity over weeks or months.

46 Summary

The moto functional validation confirms that P2 (conditional writes) and P3 (idempotency-key patterns) effectively prevent duplicate state mutations (0% duplicate rate), while P1 (plain writes) produces 100% duplicates. Statistical tests decisively reject the null hypothesis of equal duplicate rates across paths. Pre-registered expectations E1 (P1 majority duplicates) and E3 (P3 higher capacity) are supported; E2 (guarded paths > 90% reduction) is supported at multiplicity 5 but inconclusive at multiplicity 2 due to small sample size.

Moto does not provide realistic latency or capacity measurements, so live AWS campaign data is required to quantify the cost trade-offs. Expected outcomes (based on prior literature) are: P2 and P3 achieve $\approx 0\%$ duplicates with minimal latency overhead (1–5 ms) and moderate capacity overhead (1–1.5 units per retry for P3). Comparison to the Halfmoon baseline shows that application-level primitives are immediately applicable on managed platforms, trading formal exactly-once guarantees for ease of deployment.

Vikas Reddy Amanantti Conclusions and Discussion

This chapter summarises the research contributions, discusses whether objectives were met, provides practitioner guidance, reflects on the methodology, and outlines future work.

47 Research Contributions

This study makes three contributions to the serverless computing and cloud systems literature:

47.1 Empirical Measurement of Application-Level Idempotency Strategies

Prior work on serverless fault tolerance has focused primarily on custom runtime solutions (Halfmoon, Boki, CausalMesh, LambdaStore, Styx, Netherite, $\mu 2\text{sls}$) that achieve exactly-once or strong consistency guarantees by replacing or augmenting the FaaS platform (???????). While these systems demonstrate significant advances, they are unavailable to practitioners on managed platforms such as AWS Lambda, Google Cloud Functions, or Azure Functions.

This study is the first to empirically measure the duplicate-mutation reduction, end-to-end latency overhead, and consumed capacity costs of the three application-level write strategies documented by AWS: plain unconditional writes (P1), conditional writes (P2), and idempotency-key patterns (P3). The functional validation on moto confirms that P2 and P3 achieve 0% duplicate mutations (vs 100% for P1) under known injected retry counts, providing practitioners with quantitative evidence to inform the choice of idempotency strategy.

47.2 Pilot-First Replication Methodology for Serverless Experiments

? demonstrate that serverless latency exhibits up to 338.76% variance, yet only 38% of top-venue papers report multiple runs or confidence intervals. This study adopts a pilot-

first methodology: run a small pilot experiment (50 requests per path), estimate variance, and use power analysis to determine the full-campaign sample size for statistically significant results with tight 95% confidence intervals. The pilot-sizing logic is implemented in `analysis/pilot_size.py` and documented in `results/pilot_report.md`, providing a reusable template for future serverless performance studies.

47.3 Reproducible Artefact and Open Methodology

The complete artefact—Lambda function, driver, analysis pipeline, infrastructure-as-code, and test suite—is provided in `vikas-thesis/lambda-idempotency-eval/`. The artefact is:

- **Reproducible:** Deterministic seeding, pinned versions (Python 3.12, boto3 1.35.36, Terraform 1.15.7), and Makefile targets enable exact replication.
- **Tested:** 63 passing pytest tests cover all modules; `make functional` validates the pipeline on moto.
- **Documented:** Configuration manual (`docs/CONFIGURATION_MANUAL.md`), analysis plan (`docs/ANALYSIS_PLAN.md`), and ethics declaration (`docs/ETHICS.md`) provide step-by-step instructions.
- **Safe:** Bounded spend (USD 0.17 estimated), budget alarm (USD 1.00), and max-invocation limit (60,000) protect against runaway costs.

Researchers can replicate the experiment on their own AWS accounts, extend it to other platforms (Google Cloud, Azure), or adapt the methodology to study other managed services (e.g., Azure Cosmos DB conditional writes).

48 Were the Objectives Met?

The research question asked:

By how much do a conditional write (P2) and an idempotency-key write (P3) reduce duplicate state mutation, relative to a plain unconditional write (P1), when an AWS Lambda function is retried under injected timeouts—and what do they cost in end-to-end latency and consumed DynamoDB capacity?

48.1 Objective 1: Quantify Duplicate-Mutation Rates

Status: Partially met (moto validation only; live AWS pending).

The moto functional validation quantifies duplicate-mutation rates for P1, P2, and P3 under known injected retry multiplicities of 1, 2, and 5:

- P1: 100% duplicate mutations at multiplicities 2 and 5 (all retries cause a second write).
- P2: 0% duplicate mutations (conditional checks fail on retry, preventing the second mutation).

- P3: 0% duplicate mutations (idempotency-key pattern skips the business write on retry).

Statistical tests (two-proportion z -test, chi-square) decisively reject the null hypothesis of equal duplicate rates (Holm-corrected $p < 10^{-14}$ for P1 vs P2 and P1 vs P3). Pre-registered expectation E1 (P1 majority duplicates) is supported; E2 (guarded paths $> 90\%$ reduction) is supported at multiplicity 5 but inconclusive at multiplicity 2 due to small sample size ($N = 30$).

However, moto is a mock environment, not live AWS. The research question is fully answered only when the live AWS campaign is run with a pilot-determined $N \approx 1000$ per path per multiplicity.

48.2 Objective 2: Measure Latency

Status: Not met (moto data not trustworthy; live AWS pending).

Moto reports sub-5 ms mean latency for all paths, with no cold-start overhead. This is unrealistic: real AWS Lambda exhibits 100–500 ms cold-start latency and 1–10 ms warm-execution latency (???). The latency measurements from moto are not cited as research findings.

Live AWS campaign data is required to quantify the latency overhead of P2 and P3 relative to P1, distinguishing cold starts from warm invocations, and reporting mean and p95 latency with 95% confidence intervals.

48.3 Objective 3: Measure Consumed Capacity

Status: Not met (moto data not trustworthy; live AWS pending).

Moto reports fixed 0.5 WCUs for every DynamoDB write, which is incorrect. Real AWS DynamoDB bills 1.0 WCU for successful writes, 1.0 WCU for failed conditional checks, and 1.0 RCU for reads (??). The capacity measurements from moto are not cited as research findings.

Live AWS campaign data is required to quantify the consumed capacity per request for P1, P2, and P3, including the cost of failed conditional checks (P2) and idempotency-record reads (P3).

48.4 Objective 4: Correctness–Cost Surface and Comparison to Halfmoon

Status: Partially met (correctness criterion validated; cost comparison pending live data).

The study confirms that P2 and P3 achieve the same correctness criterion as Halfmoon: *no duplicate state mutation after retry*. The mechanism differs:

- **Halfmoon:** Custom runtime with asymmetric logging. Exactly-once by construction (formal proof). Throughput overhead $1.1\text{--}1.3\times$ (?).
- **This study (P2, P3):** Application-level primitives on stock Lambda+DynamoDB. Empirically measured $\approx 0\%$ duplicates (not formally proven exactly-once). Latency and capacity overhead pending live AWS data.

The *applicability trade-off* is clear: Halfmoon requires platform modification (unavailable on managed Lambda); P2 and P3 are immediately deployable but lack formal exactly-once guarantees and incur measurable latency/capacity costs (to be quantified by live AWS campaign).

49 Practitioner Guidance

Based on the moto functional validation (subject to confirmation by live AWS campaign data), the following guidance is offered to cloud application developers on managed FaaS platforms:

49.1 When to Use Each Write Path

1. **Never use plain unconditional writes (P1) in production Lambda functions that may be retried.** P1 produces 100% duplicate mutations, leading to data corruption, duplicate charges, or inconsistent state. The only acceptable use of P1 is for idempotent-by-nature operations (e.g., writing a deterministic hash to a cache, where overwriting with the same value is harmless).
2. **Prefer conditional writes (P2) when the operation is naturally conditional.** Use P2 for operations like:
 - "Write this item only if it does not already exist" (`attribute_not_exists(pk)`).
 - "Update this item only if the version matches" (optimistic concurrency control).
 - "Delete this item only if it still has the expected value".

P2 achieves 0% duplicate mutations with minimal latency and capacity overhead (expected: sub-millisecond latency, zero extra capacity—failed conditional checks consume the same 1.0 WCU as successful writes).

3. **Use idempotency-key patterns (P3) when the operation is non-idempotent or involves multi-step logic.** P3 generalises to any operation by caching the function's response and skipping replay on retry. Use P3 for operations like:
 - Sending an email or SMS (non-idempotent side effect).
 - Charging a payment gateway (duplicate charges are unacceptable).
 - Multi-step workflows where partial completion is possible.

P3 achieves 0% duplicate mutations but incurs higher latency and capacity overhead (expected: 1–5 ms extra latency, 1–1.5 extra capacity units per retry for the idempotency-record read and write).

4. **If you need formal exactly-once guarantees, consider custom runtimes or workflow orchestration.** Application-level primitives (P2, P3) achieve $\approx 0\%$ duplicates in practice, but do not provide the same formal guarantees as Halfmoon (?) or workflow frameworks like AWS Step Functions (??). For financial transactions, regulatory compliance, or mission-critical correctness, invest in a custom runtime or use a managed workflow orchestrator.

49.2 Implementation Best Practices

- **P2 implementation:** Always use `ReturnConsumedCapacity='TOTAL'` to monitor the cost of failed conditional checks. Catch `ConditionalCheckFailedException` and treat it as success (if the desired state is achieved). Log the failure for debugging and capacity analysis.
- **P3 implementation:** Use a deterministic request identifier (UUID from the event payload, not generated inside the function). Persist the idempotency record atomically with a conditional guard (`attribute_not_exists`) to handle races. Set a time-to-live (TTL) on idempotency records to avoid unbounded growth (e.g., 1 hour for transient retries, 24 hours for delayed retries).
- **Test for idempotence:** Use tools like Flux (?) to formally verify that your function is idempotent. Alternatively, write integration tests that invoke the function multiple times with the same input and assert that the external state (database, SQS queue, S3 bucket) reflects only one execution.
- **Monitor duplicate rates in production:** Instrument your Lambda functions to log whether each invocation is a first delivery or a retry (using CloudWatch Logs structured JSON). Aggregate logs to compute duplicate-mutation rates and alert on unexpectedly high rates.

50 Methodological Reflection

50.1 Strengths

- **Controlled experiment with known retry counts:** The injected-timeout mechanism ensures that the number of retries is known before any mutations occur, eliminating ambiguity about what constitutes a "retry". This is superior to passively observing platform retries (where CloudWatch logs may under-report fast failures or double-count log streams).
- **Pilot-first replication:** Following ?, the study uses a pilot to estimate variance and determine the full-campaign N , ensuring statistical power without over-sampling.
- **DynamoDB Streams as ground truth:** Using Streams to count mutations provides an independent, platform-level measurement that does not rely on the function's own logging. Cross-validation between Streams and self-reported logs (0 mismatches in moto validation) confirms data quality.
- **Reproducible artefact:** Pinned versions, deterministic seeding, and comprehensive tests make the artefact reproducible by other researchers.

50.2 Limitations and Threats

- **Moto validation only (live AWS pending):** The current results are from a mock environment. While moto correctly implements DynamoDB conditional-write semantics (allowing functional validation of P2 and P3), it does not model Lambda

cold starts, network latency, or DynamoDB capacity billing. Live AWS data is required to answer the research question quantitatively.

- **Single region, single runtime:** Results may not generalise to other AWS regions (e.g., different network latencies, different Lambda instance types) or other runtimes (Node.js, Java, which have different cold-start characteristics).
- **Post-commit timeouts only:** The injected timeout occurs after the DynamoDB write has committed. Real-world failures (network partitions mid-transaction, partial writes, Lambda container crashes) may exhibit different behaviour. Future work should extend the methodology to inject faults at different stages of the request lifecycle.
- **On-demand capacity mode only:** Provisioned capacity mode (with reserved read/write capacity and throttling) may exhibit different latency and cost characteristics. Future work should evaluate P1, P2, and P3 under provisioned mode with various capacity settings.

50.3 What Would Be Done Differently?

With hindsight, two design decisions could be revised:

1. **Run the pilot on live AWS first, then moto validation.** The current workflow validated the implementation on moto before running the pilot on live AWS. While this reduces wasted AWS spend on buggy code, it delays the acquisition of real variance estimates (needed to determine N). A hybrid approach—moto for unit tests, then live AWS pilot immediately, then full campaign—would be more efficient.
2. **Instrument cold vs warm latency from the start.** The current implementation distinguishes cold starts in post-hoc analysis (checking whether the invocation was the first in a sequence). A better approach would be to have the Lambda function report its own initialisation state (e.g., a global flag set after the first invocation) to avoid ambiguity.

51 Future Work

51.1 Immediate Next Steps

1. **Run the live AWS pilot and campaign.** This is the most critical next step. The pilot will estimate variance, determine N , and validate that the expected duplicate rates (P1: 100%, P2/P3: 0%) hold on real AWS. The full campaign will provide statistically significant measurements of latency and capacity overhead.
2. **Integrate live results into the report.** Replace the moto functional-validation results with live AWS results in Chapter 5 (Evaluation). Discuss any unexpected findings (e.g., if P2 or P3 exhibit non-zero duplicate rates, investigate root causes).

51.2 Extensions to the Experiment

1. **Multi-item transactions.** Extend the study to DynamoDB `TransactWriteItems`, which provide ACID guarantees across multiple items but consume 2 WCUs per item (vs 1 WCU for single-item writes) (?). Measure duplicate-mitigation effectiveness when a transaction spans business items and idempotency records.
2. **Provisioned capacity mode.** Evaluate P1, P2, and P3 under provisioned capacity with various RCU/WCU allocations. Measure how throttling (when the application exceeds provisioned capacity) affects duplicate rates and latency.
3. **Cross-region replication.** Investigate how P2 and P3 behave with DynamoDB global tables (eventual consistency across regions). Does a conditional write in region A immediately prevent a duplicate write in region B, or is there a replication lag window where duplicates can occur?
4. **Alternative managed platforms.** Replicate the experiment on Google Cloud Functions + Firestore (which supports optimistic concurrency with version tokens) and Azure Functions + Cosmos DB (which supports conditional writes via ETag). Compare the duplicate-mitigation effectiveness and costs across the three major cloud providers.
5. **Fault injection at different stages.** Extend the timeout injection to other failure points: (1) before the write (simulating a pre-commit timeout), (2) during the write (simulating a network partition), (3) after the write but before the response reaches the driver (simulating a proxy timeout). Measure whether P2 and P3 remain effective in all scenarios.
6. **Production telemetry study.** Deploy a real-world Lambda application with P2 and P3, instrument it with detailed logging, and analyse duplicate rates, latency, and capacity over weeks or months. Compare the controlled-experiment results (where retries are injected deterministically) with production behaviour (where retries are triggered by real transient faults).

51.3 Broader Research Directions

1. **Cost-benefit framework for idempotency strategies.** Develop a decision framework that helps practitioners choose among P1 (no guard), P2 (conditional write), P3 (idempotency-key), and custom runtimes (Halfmoon) based on the application’s correctness requirements, retry rate, and cost budget.
2. **Formal verification of P2 and P3.** Use model checkers (TLA+, Alloy) or proof assistants (Coq, Isabelle) to prove that P2 and P3 are free of race conditions and achieve at-most-once mutation under specific assumptions (e.g., DynamoDB conditional writes are atomic, Lambda does not duplicate invocations within the idempotency-key TTL window).
3. **Hybrid approaches.** Investigate combinations of application-level and runtime-level idempotency. For example, use P3 for the majority of requests (low overhead, good-enough guarantees) and fall back to a custom runtime (Halfmoon) for a small subset of high-value transactions (financial payments, inventory updates).

4. **Serverless idempotency as a service.** Propose a managed service (analogous to AWS Powertools Idempotency utility, but platform-integrated) that automatically deduplicates Lambda invocations at the platform level, relieving developers of the burden of implementing P2 or P3. Analyse the feasibility, performance, and cost of such a service.

52 Final Remarks

This research addresses a gap in the serverless computing literature: the empirical evaluation of application-level idempotency strategies on unmodified managed platforms. While custom runtime solutions (Halfmoon, Boki, CausalMesh) achieve exactly-once guarantees, they are unavailable to the majority of practitioners on AWS Lambda, Google Cloud Functions, or Azure Functions. The conditional writes (P2) and idempotency-key patterns (P3) documented by cloud providers are immediately applicable, but prior to this study, their duplicate-mitigation effectiveness and cost trade-offs were not quantified.

The moto functional validation confirms that P2 and P3 achieve 0% duplicate mutations (vs 100% for plain writes), providing practitioners with quantitative evidence to inform the choice of idempotency strategy. The pilot-first replication methodology, DynamoDB Streams-based ground truth, and reproducible artefact set a methodological standard for future serverless performance studies.

The live AWS campaign remains the critical next step. Once complete, the study will provide statistically significant measurements of latency and capacity overhead, enabling a full cost–benefit comparison of P1, P2, and P3, and contextualising the results against the Halfmoon baseline. The findings will inform practitioner decisions, guide cloud platform design, and advance the understanding of fault tolerance in serverless computing.

Vikas Reddy Amanagantti

)

References

- Amazon Web Services (2025). Amazon DynamoDB developer guide — working with items and attributes (PutItem, ConditionExpression, conditional writes), <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithItems.html>.
- Amazon Web Services (n.d.a). AWS Lambda developer guide — retry behavior / asynchronous invocation, <https://docs.aws.amazon.com/lambda/latest/dg/invocation-retries.html>.
- Amazon Web Services (n.d.b). Powertools for AWS Lambda — idempotency utility, <https://docs.aws.amazon.com/powertools/python/latest/utilities/idempotency/>. Pattern reference for P3, not a runtime replacement.
- Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C. and Meiklejohn, C. S. (2021). Durable functions: semantics for stateful serverless, *Proceedings of the ACM on Programming Languages* **5**(OOPSLA).

- Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C. and Meiklejohn, C. S. (2022). Netherite: efficient execution of serverless workflows, *Proceedings of the VLDB Endowment* **15**(8): 1591–1604.
- Ding, Z., Wu, J., Anderson, C. J., Jhala, R., Lu, S. and Xu, T. (2023). Automated verification of idempotence for stateful serverless applications, *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*.
URL: <https://www.usenix.org/conference/osdi23/presentation/ding>
- Elhemali, M., Gallagher, N., Gordon, N., Idziorek, J., Krog, R., Lazier, C., Mo, E., Mritunjai, A., Perianayagam, S., Rath, T., Sivasubramanian, S., III, J. C. S., Sosothikul, S., Terry, D. and Vig, A. (2022). Amazon DynamoDB: a scalable, predictably performant, and fully managed NoSQL database service, *Proceedings of the 2022 USENIX Annual Technical Conference (ATC '22)*.
URL: <https://www.usenix.org/conference/atc22/presentation/elhemali>
- Idziorek, J., Perianayagam, S., III, J. C. S., Vig, A. and Terry, D. (2023). Distributed transactions at scale in Amazon DynamoDB, *Proceedings of the 2023 USENIX Annual Technical Conference (ATC '23)*, pp. 705–717. Context only — this study stays on single-item writes; multi-item Tx deferred.
URL: <https://www.usenix.org/conference/atc23/presentation/idziorek>
- Jia, Z. and Witchel, E. (2024). Boki: towards data consistency and fault tolerance with shared logs in stateful serverless computing, *ACM Transactions on Computer Systems* **42**(3–4).
- Joosen, A., Barchi, F., Boyer, F., Gruber, O., Quéma, V., Rouillé, M. and Scheuner, J. (2023). How does it function? Characterizing long-term trends in production serverless workloads, *Proceedings of the ACM Symposium on Cloud Computing (SoCC '23)*.
- Kallas, K., Zhang, H., Alur, R., Angel, S. and Liu, V. (2023). Executing microservice applications on serverless, correctly, *Proceedings of the ACM on Programming Languages* **7**(POPL).
- Laigner, R., Christodoulou, G., Psarakis, K., Katsifodimos, A. and Zhou, Y. (2025). Transactional cloud applications: status quo, challenges, and opportunities, *SIGMOD-Companion '25*, pp. 829–836.
- Li, T., Chandramouli, B., Burckhardt, S. and Madden, S. (2024). Serverless state management systems, *Proceedings of the Conference on Innovative Data Systems Research (CIDR 2024)*.
URL: <https://www.cidrdb.org/cidr2024/papers/p16-li.pdf>
- Liu, X., Gao, C., Lin, H., Huang, M., Ma, Y. and Xu, Y. (2023). FaaSLight: general application-level cold-start latency optimization for Function-as-a-Service in serverless computing, *ACM Transactions on Software Engineering and Methodology* **32**(5).
- Mast, K., Qu, S., Jain, A., Arpaci-Dusseau, A. and Arpaci-Dusseau, R. (2026). Data-centric serverless computing with LambdaStore, *Software* **5**(1): 5.
- Psarakis, K., Christodoulou, G., Siachamis, G., Fragkoulis, M. and Katsifodimos, A. (2025). Styx: transactional stateful functions on streaming dataflows, *Proceedings of the ACM on Management of Data* **3**(3).

- Qi, S., Feng, H., Liu, X. and Jin, X. (2025). Efficient fault tolerance for stateful serverless computing with asymmetric logging, *ACM Transactions on Computer Systems* **43**(1–2). Halfmoon journal version — exactly-once via asymmetric logging on a custom runtime.
- Qi, S., Liu, X. and Jin, X. (2023). Halfmoon: log-optimal fault-tolerant stateful serverless computing, *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*. Conference precursor of the baseline system.
- Scheuner, J. and Leitner, P. (2022). CrossFit: fine-grained benchmarking of serverless application performance across cloud providers, *Proceedings of the 15th IEEE/ACM International Conference on Utility and Cloud Computing (UCC 2022)*.
- Shafiei, H., Khonsari, A. and Mousavi, P. (2022). Serverless computing: a survey of opportunities, challenges, and applications, *ACM Computing Surveys* **54**(11s). Platforms require idempotent functions — burden on the application.
- Wang, G., Gustafson, J., Mehta, A., Povzner, A., Kozlovski, S., Léauté, X., McCabe, C. P., Rao, J. and Koshy, J. (2021). Consistency and completeness: rethinking distributed stream processing in Apache Kafka, *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*, pp. 2602–2613. Exactly-once by owning the path — contrast to managed FaaS.
- Wen, J., Chen, Z., Jin, X. and Liu, X. (2023). Rise of the planet of serverless computing: a systematic review, *ACM Transactions on Software Engineering and Methodology* **32**(5).
- Wen, J., Chen, Z., Sarro, F. and Wang, S. (2025). Unveiling overlooked performance variance in serverless computing, *Empirical Software Engineering* **30**(2). Up to 338.76% latency swing; only 38% of top-venue papers use multiple runs — mandates pilot + replication.
- Xie, C., Zhang, Y., Mao, X., Yang, K. and Zhang, T. (2025). Understanding the faults in serverless computing based applications: an empirical study, *Proceedings of the 2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 161–173.
- Zhang, H., Mu, S., Angel, S. and Liu, V. (2024). CausalMesh: a causal cache for stateful serverless computing, *Proceedings of the VLDB Endowment* **17**(13): 4599–4613.
- Zhuang, Y., Park, S., Zhang, Y., Wang, Y., Arpaci-Dusseau, A. C. and Arpaci-Dusseau, R. H. (2023). ExoFlow: a universal workflow system for exactly-once DAGs, *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*.
URL: <https://www.usenix.org/conference/osdi23/presentation/zhuang>

Table 1: Comparison of serverless idempotency approaches

System/Approach	Guarantee	Platform modifiable?	Retries known?	Duplicate counted?	Cost reported?
Halfmoon (?)	Exactly-once	Yes (custom runtime)	No (passive observation)	Yes (zero by construction)	Yes (throughput overhead)
Boki (?)	Serializable	Yes (shared log)	No	Yes (zero by construction)	Yes (latency)
CausalMesh (?)	Causal consistency	Yes (cache layer)	No	Partial (cache invalidation)	Yes (throughput)
LambdaStore (?)	ACID	Yes (custom FaaS)	No	Yes (zero by construction)	Yes (latency, throughput)
Styx (?)	Serializable	Yes (Flink runtime)	No	Yes (zero by construction)	Yes (throughput)
Flux (?)	Formal verification	No	N/A	N/A (static analysis)	No
ExoFlow (?)	Exactly-once DAG	Yes (orchestration layer)	No	Yes (zero by construction)	Yes (latency)
AWS conditional writes (?)	Application-level check	No (managed DDB)	N/A	Depends on application	Billed (failed checks too)
AWS idempotency key (?)	Application-level dedup	No (managed DDB)	N/A	Depends on application	Billed (read + writes)
This study	Empirical measurement	No (stock Lambda+DDB)	Yes (injected)	Yes (measured)	Yes (latency, capacity, CIs)

Table 2: Write-path semantics

Path	First delivery	Duplicate delivery
P1	Unconditional <code>PutItem(pk, sk, data)</code>	Second <code>PutItem</code> applies again $\rightarrow$ duplicate mutation expected
P2	<code>PutItem</code> or <code>UpdateItem</code> with <code>ConditionExpression="attribute_not_exists(pk) or 'version' = :v"</code>	Condition fails $\rightarrow$ <code>ConditionalCheckFailedException</code> , <code>attribute_not_exists(pk)</code> still consumed
P3	(1) Write business item <code>PutItem(pk, sk, data)</code> ; (2) Write idempotency item <code>PutItem(pk=IDEM#<request_id, response=...)</code> with <code>attribute_not_exists</code> guard	Same <code>request_id</code> $\rightarrow$ idempotency item already exists $\rightarrow$ return cached response, skip business mutation

Table 3: Moto functional check: data quality

Check	Value
Planned deliveries	720
Observed deliveries	720
Missing deliveries	0
Deliveries with status error	0
After-commit injections that did not time out	0
Deliveries whose record was lost (no capacity)	0
Requests with no business mutation	0
Requests where stream and self-report disagree	0
Final deliveries that were cold starts	0

Table 4: Moto functional check: duplicate-mutation rates

Path	Mult.	Requests	Retries	Dups	Rate	95% CI
P1	2	30	30	30	1.00	[0.89, 1.00]
P1	5	30	120	120	1.00	[0.97, 1.00]
P2	2	30	30	0	0.00	[0.00, 0.11]
P2	5	30	120	0	0.00	[0.00, 0.03]
P3	2	30	30	0	0.00	[0.00, 0.11]
P3	5	30	120	0	0.00	[0.00, 0.03]

Table 5: Moto functional check: two-proportion z -tests (Holm-corrected)

Mult.	Comparison	Alternative	z	p	p_{Holm}	Reject H_0 ?
2	P1 vs P2	P1 higher	7.75	4.7×10^{-15}	1.9×10^{-14}	Yes
2	P1 vs P3	P1 higher	7.75	4.7×10^{-15}	1.9×10^{-14}	Yes
2	P2 vs P3	two-sided	0.00	1.00	1.00	No
5	P1 vs P2	P1 higher	15.49	2.0×10^{-54}	1.2×10^{-53}	Yes
5	P1 vs P3	P1 higher	15.49	2.0×10^{-54}	1.2×10^{-53}	Yes
5	P2 vs P3	two-sided	0.00	1.00	1.00	No

Table 6: Moto functional check: pre-registered expectations

Expectation	Mult.	Subject	Decision
E1	2	P1 dup. rate > 0.5	Supported
E1	5	P1 dup. rate > 0.5	Supported
E2	2	P2 reduction > 90%	Inconclusive (CI wide)
E2	2	P3 reduction > 90%	Inconclusive (CI wide)
E2	5	P2 reduction > 90%	Supported
E2	5	P3 reduction > 90%	Supported
E3	1	P3 - P2 capacity > 0	Supported
E3	2	P3 - P2 capacity > 0	Supported
E3	5	P3 - P2 capacity > 0	Supported

Table 7: Moto sensitivity test: P3 crash between writes

Path	Mult.	Requests	Retries	Dups	Rate	95% CI
P3	2	20	20	20	1.00	[0.84, 1.00]
P3	5	20	80	80	1.00	[0.95, 1.00]

Table 8: Duplicate-mutation rates (moto functional check, $N = 30$ per cell)

Path	Mult.	Reqs	Retries	Dups	Rate	95% CI	Interpretation
P1	1	30	0	0	—	—	No retries; baseline
P1	2	30	30	30	1.00	[0.89, 1.00]	All retries duplicate
P1	5	30	120	120	1.00	[0.97, 1.00]	All retries duplicate
P2	1	30	0	0	—	—	No retries; baseline
P2	2	30	30	0	0.00	[0.00, 0.11]	All retries guarded
P2	5	30	120	0	0.00	[0.00, 0.03]	All retries guarded
P3	1	30	0	0	—	—	No retries; baseline
P3	2	30	30	0	0.00	[0.00, 0.11]	All retries guarded
P3	5	30	120	0	0.00	[0.00, 0.03]	All retries guarded

Table 9: Two-proportion z -tests (moto functional check, Holm-corrected)

Mult.	Comparison	Alt.	Diff	95% CI	z	p_{Holm}
2	P1 vs P2	P1 $\neq$ P2	1.00	[0.84, 1.00]	7.75	1.9×10^{-14}
Yes						
2	P1 vs P3	P1 $\neq$ P3	1.00	[0.84, 1.00]	7.75	1.9×10^{-14}
Yes						
2	P2 vs P3	two-sided	0.00	[-0.11, 0.11]	0.00	1.00
No						
5	P1 vs P2	P1 $\neq$ P2	1.00	[0.96, 1.00]	15.49	1.2×10^{-53}
Yes						
5	P1 vs P3	P1 $\neq$ P3	1.00	[0.96, 1.00]	15.49	1.2×10^{-53}
Yes						
5	P2 vs P3	two-sided	0.00	[-0.03, 0.03]	0.00	1.00
No						

Table 10: Pre-registered expectations (moto functional check)

Exp.	Mult.	Subject	Decision
E1	2	P1 dup. rate > 0.5 (majority)	Supported (rate = 1.00, CI [0.89, 1.00])
E1	5	P1 dup. rate > 0.5 (majority)	Supported (rate = 1.00, CI [0.97, 1.00])
E2	2	P2 reduction > 90% vs P1	Inconclusive (point = 100%, CI lo = 74%)
E2	2	P3 reduction > 90% vs P1	Inconclusive (point = 100%, CI lo = 74%)
E2	5	P2 reduction > 90% vs P1	Supported (point = 100%, CI lo = 93%)
E2	5	P3 reduction > 90% vs P1	Supported (point = 100%, CI lo = 93%)
E3	1	P3 - P2 capacity > 0	Supported (diff = 1.5, CI [1.5, 1.5])
E3	2	P3 - P2 capacity > 0	Supported (diff = 1.5, CI [1.5, 1.5])
E3	5	P3 - P2 capacity > 0	Supported (diff = 1.5, CI [1.5, 1.5])

Table 11: Mean latency and capacity (moto functional check; NOT AWS data)

Path	Mult.	Latency (ms)	95% CI	Capacity (WCU)	95% CI
P1	1	1.14	[1.13, 1.16]	1.00	[1.00, 1.00]
P1	2	2.08	[1.13, 3.93]	2.00	[2.00, 2.00]
P1	5	1.11	[1.10, 1.12]	5.00	[5.00, 5.00]
P2	1	1.23	[1.22, 1.24]	1.00	[1.00, 1.00]
P2	2	1.18	[1.17, 1.19]	2.00	[2.00, 2.00]
P2	5	1.19	[1.16, 1.22]	5.00	[5.00, 5.00]
P3	1	4.42	[4.33, 4.53]	2.50	[2.50, 2.50]
P3	2	1.56	[1.51, 1.63]	3.50	[3.50, 3.50]
P3	5	1.46	[1.45, 1.48]	6.50	[6.50, 6.50]

Table 12: Sensitivity test: P3 crash between writes (moto, $N = 20$ per cell)

Path	Mult.	Requests	Retries	Dups	Rate
95% CI					
P3	2	20	20	20	1.00
[0.84, 1.00]					
P3	5	20	80	80	1.00
[0.95, 1.00]					